AI 大模型应用开发 V3.0

综合项目实战 · Day 02

AI 鉴品师

多模态智能评分与竞品深度解剖

四维评分引擎 + Vision 多模态图片分析 + 竞品差评挖掘

技术栈：N8N + OpenClaw + 通义千问 API + 千问 VL 多模态 + 飞书 AI 字段

上海博为峰教育 · 教研三部

目录

前置准备：N8N Execute Command 环境配置（Windows npm 版）

实验五：四维评分引擎（90 分钟）

实验六：Vision 多模态商品图片分析（90 分钟）★ 重点实验

实验七：竞品深度解剖（60 分钟）

实验八：N8N 工作流扩展——接续 Day01 串联全链路（45 分钟）

实验九：飞书 AI 快捷字段配置（30 分钟）

前置准备-01、为 raw_products 表补充 Vision 相关字段

后面实验六的工作流多个节点会用到 raw_products 表中的 vision_analyzed、vision_score、vision_data 三个字段，但 Day01 实验一建表时并没有这三个字段（当时还没规划 Vision 分析），因此在正式搭建本实验工作流之前，需要先对 raw_products 表做一次 ALTER 补字段操作。

为什么需要这些字段？

vision_analyzed 是一个状态标记字段，用来记录某个商品的图片是否已经被 Vision 模型分析过。它的取值为 0（未分析，默认值）或 1（已分析）。Vision 多模态分析慢且贵——每张图片调用 qwen-vl-max 需要 3~10 秒，按 token 计费每张图约几分钱。如果不加这个标记，每次跑 N8N 工作流都会重复分析所有商品，已经分析过的也会再花一次钱、再等一次时间。

有了这个标记字段后，工作流逻辑就变成：先查 vision_analyzed = 0 的商品（只取需要分析的）→ 调用 Vision API 分析 → 完成后 UPDATE 该商品的 vision_analyzed = 1 → 下次工作流自动跳过它。这就是「增量处理」的核心思想：只处理新数据，避免重复劳动。

另外两个字段的用途：vision_score 存储 Vision 给出的视觉评分（DECIMAL 类型，用于排序和筛选）；vision_data 存储 Vision 返回的完整 JSON（包含颜色、风格、受众等结构化结果，供飞书看板展示和详细分析使用）。

1）执行 ALTER TABLE 补 Vision 标志相关字段

在 MySQL 客户端中执行以下 SQL，一次性把三个字段加上：

```
ALTER TABLE raw_products
ADD COLUMN vision_analyzed TINYINT DEFAULT 0
    COMMENT '是否已做 Vision 分析：0=未分析，1=已分析',
ADD COLUMN vision_score DECIMAL(3,1)
    COMMENT 'Vision 视觉评分(0-10)',
ADD COLUMN vision_data JSON
    COMMENT 'Vision 返回的完整 JSON 结果';
```

执行后所有已存在的商品 vision_analyzed 都会默认为 0，正好可以被本实验的工作流捞起来分析。验证字段是否添加成功：

```
DESC raw_products;
SELECT asin, vision_analyzed, vision_score, vision_data
FROM raw_products LIMIT 3;
```

能看到 vision_analyzed 列全部为 0，vision_score 和 vision_data 全部为 NULL，说明字段已添加成功，但还未填入分析结果——这正是工作流要做的事情。

2）再次执行 ALTER TABLE 补 Vision 解析后的字段

图片解析成功后，会识别出 **主色调、视觉风格标签、目标人群、核心卖点等信息**，这些也可以保存到 raw_products 表中，在 MySQL 客户端中执行以下 SQL，一次性再加四个字段：

```
ALTER TABLE raw_products
  ADD COLUMN vision_colors VARCHAR(255) NULL COMMENT 'Vision 识别出的主色调',
  ADD COLUMN vision_style VARCHAR(100) NULL COMMENT 'Vision 识别出的视觉风格标签',
  ADD COLUMN vision_audience VARCHAR(255) NULL COMMENT 'Vision 识别出的目标人群',
  ADD COLUMN vision_selling_pt TEXT NULL COMMENT 'Vision 识别出的核心卖点'
```

3）再次执行 ALTER TABLE 补差评挖掘相关的字段

```
ALTER TABLE raw_products
ADD COLUMN pain_points TEXT COMMENT '差评核心痛点',
ADD COLUMN pain_summary VARCHAR(500) COMMENT '差评一句话总结';
```

前置准备-02：N8N Execute Command 环境配置（Windows npm 版）

⏱ **预计时长：**30 分钟

🎯 **适用范围：**Windows 系统 + npm 方式安装的 N8N 2.0.x，实验五 5.3 / 5.4 的 Python 脚本通过 Execute Command 节点调用

📋 **解决问题：**Execute Command 节点默认禁用、Python 调用、中文乱码、stdout 字段解析

实验五的 5.3（LLM 市场风险分）和 5.4（四维加权合成）需要通过 N8N 的 Execute Command 节点调用 Python 脚本。在 Windows 环境下，这个节点默认被禁用，且存在编码等问题。本节集中解决所有前置障碍，确保后续实验顺利进行。

一、解除 Execute Command 节点禁用

> ⚠ **重要**
> 从 N8N 2.0 开始，Execute Command 节点因安全风险被默认禁用。搜索节点面板时找不到它是正常现象，需要手动配置环境变量解禁。

解决方案：通过 Windows 系统环境变量配置 NODES_EXCLUDE=[] 解除禁用，同时配置 .py 脚本访问白名单。设置完成后所有 cmd 窗口中启动的 N8N 都会生效，一次配置长期有效。

步骤 1：打开系统环境变量设置面板

方式一：按 Win 键搜索 “环境变量”，点击 “编辑系统环境变量” → 在弹出窗口右下角点 “环境变量(N)...” 按钮。

方式二：右键 “此电脑” → 属性 → 高级系统设置 → 环境变量。

步骤 2：在【系统变量】区域新建两个变量

在下方的 “系统变量(S)” 区域点击 “新建(W)...” 按钮，依次添加以下两个变量：

变量 1 — NODES_EXCLUDE

变量名：NODES_EXCLUDE

变量值：[]

作用：清空 N8N 节点禁用列表，使 Execute Command 节点重新可用。

变量 2 — N8N_RESTRICT_FILE_ACCESS_TO

变量名：N8N_RESTRICT_FILE_ACCESS_TO

变量值：D:\n8ndata;C:\Users\admin\.n8n;D:\n8n-scripts

作用：配置 N8N 可访问的文件路径白名单，多个路径用分号分隔。其中 D:\n8n-scripts 是后续 Python 脚本统一存放目录，必须包含在白名单中。

步骤 3：确认配置生效

全部配置完成后，点 “确定” 关闭所有对话框，关闭已打开的 cmd 窗口（环境变量变更不会影响已存在的窗口）。

重新打开一个新的 cmd 窗口，执行以下命令验证环境变量已生效：

```
echo %NODES_EXCLUDE%
echo %N8N_RESTRICT_FILE_ACCESS_TO%
```

能正确输出 [] 和路径白名单，说明配置已生效。然后执行 n8n start 启动 N8N，进入 N8N 工作流界面，添加节点时即可搜索到 Execute Command。

> 📌 **说明**
> N8N_RESTRICT_FILE_ACCESS_TO 是文件路径白名单，只有配置在这里的目录下的 .py 脚本才能被 N8N 访问，这是 .py 文件存放路径必须配置的变量。如果同时存在 N8N_FILE_ACCESS_WHITELIST 和 N8N_RESTRICT_FILE_ACCESS_TO 两个变量，以 N8N_RESTRICT_FILE_ACCESS_TO 为准。

二、Python 环境验证与依赖安装

Execute Command 节点在 Windows 下调用的是系统 cmd.exe，因此需要 Python 已安装并加入系统 PATH。

验证 Python 可用

```
python --version
```

能输出版本号（如 Python 3.13.3）即可。如果报 “不是内部或外部命令”，需重装 Python 并勾选 Add Python to PATH。

安装脚本所需依赖包

```
pip install requests pymysql pandas
```

> ⚠ **注意**
> Windows 下 N8N 调用脚本使用 python 命令（不是 python3），依赖包需安装在系统 Python 环境中，而非虚拟环境。

三、.py 脚本存放规范

所有 Python 脚本必须放在 N8N_RESTRICT_FILE_ACCESS_TO 配置的目录下，否则 N8N 会拒绝访问。推荐统一存放在：

```
D:\n8n-scripts\
```

脚本规划：scoring_engine.py 对应实验五 5.3 / 5.4 四维评分引擎；vision_analyzer.py 对应实验六

Vision 多模态图片分析；competitor_analyzer.py 对应实验七竞品差评分析。

四、Execute Command 节点配置方式

在 N8N 工作流中添加 Execute Command 节点，参数配置如下：

Command：python D:\n8n-scripts\scoring_engine.py

Execute Once：开启（打开开关）

> 📌 **Windows 与 Linux 命令差异**
> 教程中其他章节（如实验八）写的是 python3 /opt/scripts/scoring_engine.py，这是 Linux 写法。Windows 下请改为：python D:\n8n-scripts\scoring_engine.py（用 python 而非 python3，路径用反斜杠）。

五、从 Execute Command 输出中提取字段

Execute Command 节点的输出结构固定为三个字段：

exitCode：0 = 成功，非 0 = 脚本报错

stderr：脚本错误信息（正常为空）

stdout：脚本 print() 输出的内容（字符串）

.py 脚本约定用 print(json.dumps(result)) 输出结果，stdout 是一个 JSON 字符串。后续节点用 Edit Fields 节点的表达式模式解析：

```
{{ JSON.parse($json.stdout).字段名 }}
```

例如取 message 字段：

```
{{ JSON.parse($json.stdout).message }}
```

六、解决 stdout 中文乱码问题

Windows cmd 默认编码是 GBK，Python 输出 UTF-8，两者不一致会导致中文乱码。在每个 .py 脚本顶部加入以下代码强制 UTF-8 输出：

```
# -*- coding: utf-8 -*-
import sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
```

此后脚本中的 print() 输出的中文在 N8N 的 stdout 中将正常显示。

七、环境验证：用 test.py 测试全链路

在正式编写 scoring_engine.py 前，先用一个最简单的测试脚本验证整个调用链路是否通畅。

步骤 1：创建测试脚本 D:\n8n-scripts\test.py

```
# -*- coding: utf-8 -*-
import sys, io, json, datetime
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

args = sys.argv[1:] if len(sys.argv) > 1 else ["无参数"]
result = {
    "status": "success",
    "message": "Python 脚本调用成功！",
    "input_args": args,
    "python_version": sys.version.split(" ")[0],
    "run_time": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
}
print(json.dumps(result, ensure_ascii=False))
```

> 如果出现中文乱码，还有一种方案是：print(json.dumps(result, ensure_ascii=True))

步骤 2：在 cmd 中手动验证脚本可运行

```
python D:\n8n-scripts\test.py hello n8n
```

预期输出：

```
{"status": "success", "message": "Python 脚本调用成功！", ...}
```

步骤 3：在 N8N 中配置 Execute Command 节点

Command：python D:\n8n-scripts\test.py hello n8n

Execute Once：开启

步骤 4：判断执行结果

执行后查看输出，三项指标全部满足即代表成功：exitCode = 0（脚本正常执行）；stderr 为空（无报错）；stdout 包含完整 JSON 字符串（输出正常）。

步骤 5：用 Edit Fields 节点解析 stdout

在 Execute Command 后接 Edit Fields 节点，添加字段 message，值填表达式：

```
{{ JSON.parse($json.stdout).message }}
```

右侧输出显示 “Python 脚本调用成功！” 即代表全链路打通。

八、常见报错排查速查表

python is not recognized：Python 未加入 PATH。解决：重装 Python 勾选 Add to PATH。

file access restricted：脚本路径不在白名单。解决：检查 N8N_RESTRICT_FILE_ACCESS_TO=D:\n8n-scripts。

No such file or directory：路径写错或文件不存在。解决：检查脚本文件是否在 D:\n8n-scripts\。

exitCode: 1 + stderr 有内容：Python 脚本本身报错。解决：查看 stderr 字段里的具体错误信息。

Execute Command 节点找不到：节点未解禁。解决：确认 Windows 系统环境变量中 NODES_EXCLUDE 已设置为 [] 且 cmd 窗口已重新打开。

✅本节交付物

① Windows 系统环境变量 NODES_EXCLUDE 和 N8N_RESTRICT_FILE_ACCESS_TO 配置完成；

② D:\n8n-scripts\test.py 测试脚本；

③ N8N Execute Command 节点调用 Python 全链路验证通过；

④ 理解了 stdout 解析方式和中文编码处理方案。

实验五：四维评分引擎

- **时长**：90 分钟
- **目标**：给每个商品打一个“值不值得卖”的综合分数
- **前置条件**：Day01 的 MySQL 中已有 500+ 条商品数据

核心教学点

不是所有事情都需要大模型。前三个维度用 SQL/Python 规则计算就够了，只有「需要常识判断」的市场风险维度才需要调用 LLM。这是企业落地的核心原则：能用规则解决的不用 AI，既省钱又稳定。

说明

这个利润模型是简化版，真实项目中需要查询 1688 采购价和 FBA 费用计算器。但对于教学演示，这个比例已经足够说明问题。

面试话术

“为什么前三维不用 LLM？”——“因为前三维有确定的计算公式，用规则更稳定、更快、更便宜。只有「市场风险」需要常识推理（季节性、专利等），这才是大模型的用武之地。”

☑ 本实验交付物

- 全部商品有了四维评分和综合得分
- 可按综合得分排序查看 TOP20
- 理解了“规则 + LLM 混合”的评分架构

5.1 评分模型设计：为什么是四个维度？

- 这一节先不急着写公式，先回答一个最关键的问题：为什么是四个维度？因为选品不是“看到一个商品就拍脑袋下结论”，而是要依次回答四个业务问题：这个商品有多火？市场有多卷？能赚多少钱？会不会踩雷。
- 结合当前这批数据，7 个候选商品里有 4 个能在 BSR Top100 里找到排名，3 个找不到。这里的“未入榜”不是坏数据，而是一个很重要的业务信号：说明它们在当前榜单里热度不够强，但仍然可以参与分析。
- 所以四维评分的目标不是“把所有商品一次性排个高低”，而是把每个问题拆开回答，再把答案合起来。四个维度分别是：趋势热度分、竞争度分、利润空间分、市场风险分。
- 这里有一个统一约定：前三维都属于“越高越好”的正向分数；第四维原始输出是风险分，但在落表时会先转成安全分，方便最终总分统一成“分数越高越值得卖”。

5.2 前三维评分实现（分步教学）

在这一步里，我们会先完成三件事：

①把 raw_products、bsr_top50、product_reviews 拼成一张评分底表；

②计算趋势热度分；③计算竞争度分和利润空间分。

5.2.1 第一步：先把三张表拼成评分底表

为什么要先做底表？因为原始表是“分散的”：

raw_products 只有商品基础信息

bsr_top50 只有榜单信息

product_reviews 只有评论明细。

真正做评分时，我们需要的是“一行一个商品”的统一视图。这一步做完以后，后面所有公式都只看这张底表，不再来回跨表查数据。这样一来，SQL 更短、更稳，也更方便教学。




```
/* ============================================================
   第一步：先把三张表拼成评分底表(只统计今日的)
   目标：让每个 ASIN 只保留一行，后面的评分只看这张表
   ============================================================ */
DROP TABLE IF EXISTS score_base;
CREATE TABLE score_base AS
SELECT
    rp.asin,
    COALESCE(rp.title, bs.product_name) AS product_name,
    COALESCE(rp.price, bs.price) AS price,
    COALESCE(rp.brand, 'Unknown') AS brand,
    COALESCE(rp.category, 'Unknown') AS category,
    COALESCE(rp.crawl_date, bs.crawl_date) AS crawl_date,
    COALESCE(rp.image_url, bs.image_url) AS image_url,
    bs.bsr_rank,
    CASE
        WHEN bs.bsr_rank IS NULL THEN '未入榜'
        ELSE '已入榜'
    END AS bsr_status,
    COALESCE(bs.review_count, ra.review_count, 0) AS review_count,
    COALESCE(bs.rating, ra.avg_rating) AS rating
FROM raw_products rp
LEFT JOIN bsr_top50 bs
    ON rp.asin = bs.asin
    AND bs.crawl_date = CURDATE()          -- ← 改动①：只 JOIN 今天的榜单快照
LEFT JOIN (
    SELECT
        asin,
        COUNT(*) AS review_count,
        ROUND(AVG(rate), 1) AS avg_rating
    FROM product_reviews
    WHERE crawl_date = CURDATE()           -- ← 改动②：只聚合今天的评论（如有 crawl_date 字段）
    GROUP BY asin
```

```
) ra
    ON rp.asin = ra.asin
WHERE rp.crawl_date = CURDATE();            -- ← 改动③：主表只取今天采集的商品
```

备注：*这段 SQL 里最值得理解的函数是 COALESCE 和 LEFT JOIN。COALESCE 的意思是“从左到右找第一个不是 NULL 的值”，它本质上是一个兜底函数。LEFT JOIN 的意思是“左表全保留，右表能匹配就补信息，匹配不到也不丢数据”。*

验证方式很简单：先看总行数，再看已入榜和未入榜的数量。如果底表是对的，你应该看到 7 行候选商品，其中 4 行已入榜、3 行未入榜。

```
/* 验证底表是否拼对 */
SELECT
    COUNT(*) AS total_rows,
    SUM(bsr_status = '已入榜') AS ranked_rows,
    SUM(bsr_status = '未入榜') AS unranked_rows
FROM score_base;

SELECT
    asin,
    product_name,
    bsr_rank,
    bsr_status,
    review_count,
    rating
FROM score_base
ORDER BY bsr_status, bsr_rank;
```

如果你看到的结果确实是 7 行，并且有 4 个已入榜、3 个未入榜，那就说明底表搭好了。这里的解读不是“谁高谁低”，而是“数据口径已经统一，可以继续往下算分”。

5.2.2 第二步：趋势热度分（BSR + 评论）

- 趋势热度分回答的是第一个问题：这个商品有多火？我们把它拆成两个信号来看：BSR 排名和评论数。BSR 越小，说明排名越靠前；评论数越多，说明真实销量和市场关注度通常越强。
- 但评论数不能直接拿来打分，因为你的数据是典型长尾分布：少的只有几条，多的能到几万条。如果直接线性拉伸，低评论商品会被压得几乎没有区别，所以这里要用“对数归一化”。
- 你可以把对数归一理解成“压缩差距”。比如评论数从 10 到 100、从 100 到 1000、从 1000 到 10000，都是“增加了 10 倍”。对数会把这种“数量级变化”映射成比较接近的分数变化，而不是让 10000 条评论把 10 条评论直接碾压掉。
- 如果按这套公式去看，评论数 1、10、100、1000、10000，大致会得到 7.5、26.0、50.2、75.0、100 分。这个结果非常适合选品场景，因为我们更关心“层级差异”，而不是每一条评论的绝对值。

```
/* ============================================================
   第二步：趋势热度分
   目标：先算出 BSR 子分、评论热度子分，再合成 trend_score
   规则：
   1) 已入榜：rank 越小，分越高；rank 1 = 100 分，rank 100 = 0 分
   2) 未入榜：给一个明显惩罚分 20 分，让它能参与评分但不和榜内商品平起平坐
   3) 评论数：用对数归一化，避免长尾数据把分数拉得过于夸张
   ============================================================ */
DROP TABLE IF EXISTS score_trend;
CREATE TABLE score_trend AS
SELECT
    b.*,

    /* BSR 子分：把 1~100 的排名映射到 0~100 分 */
    CASE
        WHEN bsr_rank BETWEEN 1 AND 100 THEN
            ROUND((100 - bsr_rank) / 99 * 100, 1)
        ELSE
            20.0
    END AS bsr_score,

    /* 评论热度子分：对数归一化，评论数越多分越高，但增长会越来越慢 */
    ROUND(
        LEAST(LOG10(review_count + 1) / LOG10(10000) * 100, 100),
        1
    ) AS review_score,

    /* 趋势热度综合分：BSR 占 60%，评论热度占 40% */
    ROUND(
        (
            CASE
                WHEN bsr_rank BETWEEN 1 AND 100 THEN
                    ROUND((100 - bsr_rank) / 99 * 100, 1)
                ELSE
                    20.0
            END
        ) * 0.6
        + LEAST(LOG10(review_count + 1) / LOG10(10000) * 100, 100) * 0.4,
        1
    ) AS trend_score
FROM score_base b;
```

备注：这里又出现了几个很常见的 SQL 函数：LOG10、LEAST、ROUND。LOG10 的作用是把“原始数字”变成“数量级”；LEAST 的作用是取最小值，这里是给分数封顶，保证不超过 100；ROUND 的作用是保留一位小数，方便阅读和展示。

验证时不要只看一个商品，最好把 top10 全部拉出来看。你会发现已入榜商品通常 trend_score 更高，未入榜商品也不是零分，而是被固定压低到了一个合理的保底区间。

```
/* 验证趋势热度分 */
SELECT
    asin,
    product_name,
    bsr_rank,
    bsr_status,
    review_count,
    bsr_score,
    review_score,
    trend_score
FROM score_trend
ORDER BY trend_score DESC
LIMIT 10;
```

怎么解读这个结果？如果某个商品 BSR 排名很靠前，它的 bsr_score 会很高；如果某个商品评论数明显更多，它的 review_score 也会更高。trend_score 就是把这两个信号一起考虑后的综合热度。

5.2.3 第三步：竞争度分（评论越多，竞争越强）

- 竞争度分回答的是第二个问题：这个市场有多卷？直觉上，评论越多往往说明市场参与者越多、成熟度越高、竞争越激烈。所以这一步要反过来算分：评论越多，竞争度分越低。

- 这一步不需要复杂模型，线性反向计分就足够教学使用：评论少，得分高；评论多，得分低；如果评论数特别夸张，就把分数压到 0，不让它出现负数。

```
/* ============================================================
   第三步：竞争度分
   目标：评论越多，市场越拥挤，所以竞争度分越低
   处理方法：线性反向计分 + GREATEST 保底到 0
   ============================================================ */
DROP TABLE IF EXISTS score_competition;
CREATE TABLE score_competition AS
SELECT
    t.*,

    /*
      逻辑解释：
      - review_count = 0 时，competition_score 接近 100
      - review_count = 5000 时，competition_score 接近 0
      - 如果算出来是负数，就用 GREATEST 强制拉回 0
    */
    ROUND(
        GREATEST(100 - (review_count / 5000.0) * 100, 0),
        1
    ) AS competition_score
FROM score_trend t;
```

备注：

1、GREATEST 的意思是“取最大值”。这里用它的目的不是让分数变高，而是防止公式继续往下算时出现负数。你可以把它理解成“下限保护”。

2、把“评论很多已经很卷了”这个业务判断，换成一个容易理解的分数拐点。这里 5000 代表是评论的中位数，相当于公式里的归一化基准点。5000 条评论，市场已经非常成熟、非常拥挤，所以竞争度分压到 0

验证时建议同时看低评论商品和高评论商品。*低评论商品应该更接近 100 分，高评论商品应该更接近 0 分。这样一看就知道公式方向是对的。*

```
/* 验证竞争度分 */
SELECT
    asin,
    product_name,
    review_count,
    competition_score
FROM score_competition
ORDER BY review_count ASC
LIMIT 5;

SELECT
    asin,
    product_name,
    review_count,
    competition_score
FROM score_competition
ORDER BY review_count DESC
LIMIT 5;
```

这一步的结果很好解释：评论少，不代表一定好卖，但至少说明这个市场没有那么拥挤；评论多，不一定差，但通常意味着后来者更难切进去。

5.2.4 第四步：利润空间分（售价越高，空间越大）

- 利润空间分回答的是第三个问题：这个商品能不能赚钱？在真实跨境电商里，利润计算要考虑采购价、运费、FBA 费用、平台佣金、退货损耗等很多项。课堂里如果一上来就算完整利润，学生会很容易迷路，所以这里先用一个简化模型。
- 我们的简化思路是：价格带打分。

这个最适合教学，也最符合选品直觉。

思路是：

✓ 太低价，利润不够

✓ 中间价位最好

✓ 太高价，转化和库存压力上升，分数下降

- SQL 语句：

```
DROP TABLE IF EXISTS score_profit;
CREATE TABLE score_profit AS
SELECT
    x.*,
    CASE
        WHEN x.price IS NULL OR x.price <= 0 THEN 0
        WHEN x.price < 80  THEN 20
        WHEN x.price < 150 THEN 60
        WHEN x.price < 300 THEN 100
        WHEN x.price < 500 THEN 70
        ELSE 40
    END AS profit_score
FROM (
    SELECT
        t.asin,
        t.product_name,
        COALESCE(NULLIF(rp.price, 0), NULLIF(bs.price, 0), 0) AS price,
        t.brand,
        t.category,
        t.crawl_date,
        t.image_url,
        t.bsr_rank,
        t.bsr_status,
        t.review_count,
        t.rating,
        t.bsr_score,
        t.review_score,
        t.trend_score,
        t.competition_score
    FROM score_competition t
    LEFT JOIN raw_products rp
        ON t.asin = rp.asin
        AND rp.crawl_date = CURDATE()      -- ← 加这行
    LEFT JOIN bsr_top50 bs
        ON t.asin = bs.asin
        AND bs.crawl_date = CURDATE()      -- ← 加这行
) x;
```

计算前，先统一商品售价来源：优先从 raw_products.price 取价格；如果主表没有有效价格，再通过 ASIN 到 bsr_top50.price 中补充；如果两张表都没有价格，就按 0 处理。

统一价格后，再按价格区间打分：

```
price <= 0 -> 0 分
```

```
price < 80 -> 20 分
price < 150 -> 60 分
price < 300 -> 100 分
price < 500 -> 70 分
price >= 500 -> 40 分
```

这样设计的含义是：

低价商品虽然更容易成交，但利润空间有限，所以分数较低

```
150-300 美元通常是更理想的价格带，既有客单价，也相对容易承接利润
300-500 美元仍有利润空间，但转化压力开始上升，所以分数回落
500 美元以上虽然单价高，但购买决策更重、库存压力更大，所以不再给高分
```

这里的 COALESCE(NULLIF(...), NULLIF(...), 0) 是为了避免价格为 0 时被当作有效价格。NULLIF(price, 0) 会先把 0 转成 NULL，然后 COALESCE 才能继续往后寻找下一个可用价格。

```
/* 验证利润空间分 */
SELECT
    asin,
    product_name,
    price,
    profit_score
FROM score_profit
ORDER BY price DESC
LIMIT 10;
```

这一步的解读也很简单：我们不是在算真实净利润，而是在给“售价所带来的利润潜力”打一个教学分。后面如果要接真实利润模型，可以把这里替换成更细的成本公式。

无代码方案：把前三维封装成 N8N 子工作流（sub_rule_score）

📌 适用人群与说明

没有 SQL 客户端操作经验、希望全程在 N8N 里跑通的学员，可直接使用本节工作流，效果与前面在客户端逐段执行完全一致。

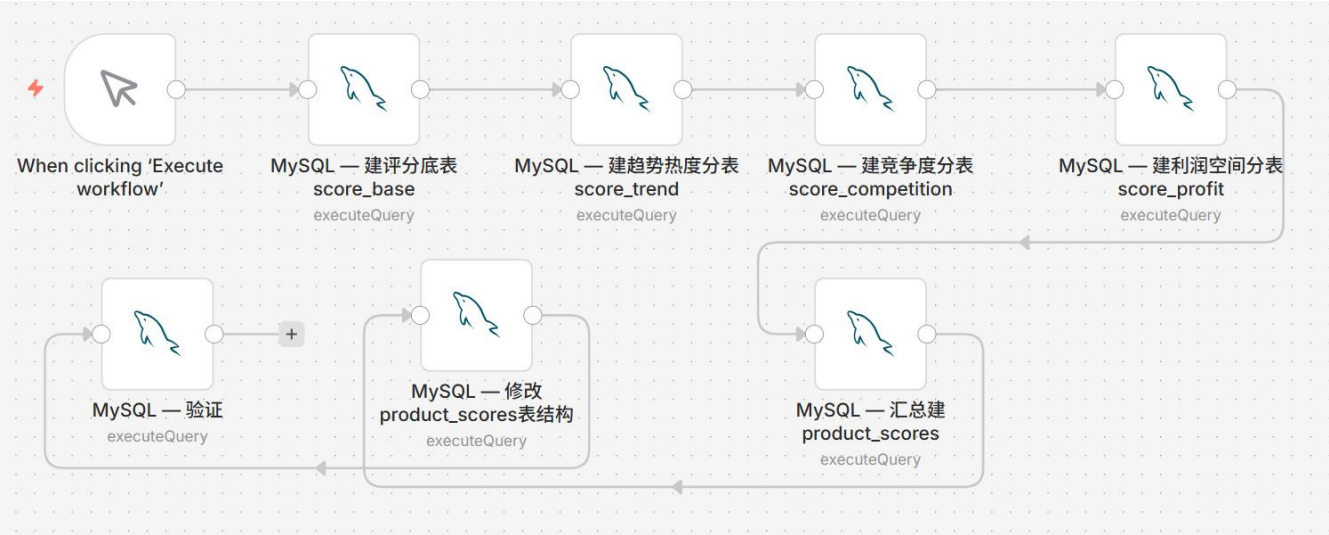
前三维的四张评分表（score_base / score_trend / score_competition / score_profit）以及 product_scores 已全部改为真实表，跨 N8N 的 MySQL 节点连接依然可见，因此可以拆成多个 MySQL 节点依次执行。

本子工作流统一命名为 sub_rule_score，最终产出 product_scores（rule_score_80 已填好，risk_score、total_score 暂为 NULL），正好作为 5.3 风险评分工作流的输入。实验八可用 Execute Workflow 节点直接引用 sub_rule_score。

工作流总览（01：sub_rule_score —— 三维评分实现，共 7 个节点）

```
[Manual Trigger]
       ↓
[节点 1: MySQL] 建评分底表 score_base
       ↓
[节点 2: MySQL] 建趋势热度分表 score_trend
       ↓
[节点 3: MySQL] 建竞争度分表 score_competition
       ↓
[节点 4: MySQL] 建利润空间分表 score_profit
       ↓
```

```
[节点 5: MySQL] 汇总建 product_scores（写入 rule_score_80）
        ↓
[节点 6: MySQL] 修改 product_scores 表结构（字段类型转 DECIMAL/TEXT）
        ↓
[节点 7: MySQL] 验证：核对行数与空值列
```



节点 1：MySQL — 建评分底表 score_base

节点类型：MySQL（Execute Query 操作）。把 5.2.1 的整段建表 SQL（已含 DROP TABLE IF EXISTS score_base; 与 CREATE TABLE score_base AS …）原样粘进 Query 框即可。因为现在是真实表，本节点执行完后，后续每个 MySQL 节点都能直接查到 score_base。

节点 2：MySQL — 建趋势热度分表 score_trend

节点类型：MySQL（Execute Query 操作）。粘入 5.2.2 的建表 SQL（DROP TABLE IF EXISTS score_trend; + CREATE TABLE score_trend AS … FROM score_base b;）。它依赖节点 1 产出的 score_base，所以必须排在节点 1 之后。

节点 3：MySQL — 建竞争度分表 score_competition

节点类型：MySQL（Execute Query 操作）。粘入 5.2.3 的建表 SQL（… FROM score_trend t;）。依赖节点 2 产出的 score_trend。

节点 4：MySQL — 建利润空间分表 score_profit

节点类型：MySQL（Execute Query 操作）。粘入 5.2.4 的建表 SQL（… FROM score_competition t）。依赖节点 3，它是前三维计算链路的最后一张中间表。

节点 5：MySQL — 汇总建 product_scores

接下来，这一步把前三维合成 rule_score_80（满分 80），risk_score、risk_summary、total_score 先留 NULL，等 5.3 的风险评分工作流回填。

节点类型：MySQL（Execute Query 操作）。

```
/* ============================================================
   先把前三维的结果整理进 product_scores
   说明：
   - rule_score_80 = 趋势热度 30% + 竞争度 25% + 利润空间 25%
   - risk_score 先留空，等 LLM 风险结果回来后再写入
   - total_score 也先留空，最后一步统一计算
   ============================================================ */
DROP TABLE IF EXISTS product_scores;
CREATE TABLE product_scores AS
SELECT
    asin,
    product_name,
    brand,
    category,
    price,
 /* 价格档位：从一次性 GROUP BY 升级为商品的固有属性 */
    CASE
        WHEN price < 150 THEN '$0-150（低端）'
        WHEN price < 250 THEN '$150-250（中低端）'
        WHEN price < 400 THEN '$250-400（中端）'
        WHEN price < 600 THEN '$400-600（中高端）'
        ELSE '$600+（高端）'
    END AS price_band,
    crawl_date,
    bsr_rank,
    bsr_status,
    review_count,
    rating,
    bsr_score,
    review_score,
    trend_score, -- 趋势热度分
    competition_score, -- 竞争度分
    profit_score, -- 利润空间分

    /* 三个规则维度的合成分：满分 80 */
    ROUND(
        trend_score * 0.30
        + competition_score * 0.25
        + profit_score * 0.25,
        1
    ) AS rule_score_80,

    NULL AS risk_score,
    NULL AS risk_summary,
    NULL AS total_score
FROM score_profit;
```

这一步的验证重点不是分数高低，而是看表结构是否已经完整：前三维都在，风险分和总分先是空值。这样做的好处是，后面只需要把 LLM 的结果回写进去即可。

```
/* 验证结果表是否建立成功 */
SELECT
    COUNT(*) AS total_rows,
    SUM(risk_score IS NULL) AS risk_score_null_rows,
    SUM(total_score IS NULL) AS total_score_null_rows
FROM product_scores;

SELECT
    asin,
    product_name,
    rule_score_80,
    risk_score,
    total_score
FROM product_scores
ORDER BY rule_score_80 DESC
LIMIT 10;
```

节点 6：MySQL — 修改 product_scores 表结构

节点类型：MySQL（Execute Query 操作）。

Query：

```
ALTER TABLE product_scores
MODIFY COLUMN rule_score_80 DECIMAL(10,1) NULL,
MODIFY COLUMN risk_score DECIMAL(10,1) NULL,
MODIFY COLUMN risk_summary TEXT NULL,
MODIFY COLUMN total_score DECIMAL(10,1) NULL;
```

节点 7：MySQL — 验证

节点类型：MySQL（Execute Query 操作）。Query：

```
SELECT COUNT(*) AS total_rows,
       SUM(rule_score_80 IS NOT NULL) AS rule_filled,
       SUM(risk_score IS NULL)        AS risk_pending
FROM product_scores;
```

rule_filled 等于总行数、risk_pending 也等于总行数，说明前三维已算好、风险分待 5.3 回填，sub_rule_score 跑通。

执行与验证

① 保存工作流并命名为 sub_rule_score；② 点 Test Workflow，按节点 1→6 顺序逐个观察 MySQL 节点是否成功；③ 因为五张表都加了 DROP TABLE IF EXISTS，本工作流可以反复执行而不会报「表已存在」，每跑一次就是用最新数据重算一遍。

5.3 第四维：LLM 市场风险分（20% 权重）

为什么第四维不能只靠 SQL？因为“有没有季节性风险”“会不会碰到专利或侵权”“这个品类会不会特别容易退货”这类问题，很难用固定规则写死。它们需要常识判断，而这正是大模型最擅长的地方。

所以第四维不再只看数字，而是让 LLM 根据商品名称、类目和价格做一个综合判断。模型输出的是 total_risk_score，数值越高说明风险越大；为了和前面“分数越高越好”的规则保持一致，我们再把它转成 safety_score，也就是安全分。

5.3.1 风险分为什么交给大模型

如果你只用 SQL 规则，最多能算出“价格高不高、评论多不多、排名前不前”，但很难判断“这个商品会不会因为专利、季节、合规问题踩雷”。这些问题对跨境电商选品非常重要，所以第四维必须交给 LLM。

5.3.2 风险评分函数

> D:/n8n-scripts/scoring_engine.py

```
import os
import sys
import json
import requests


def get_risk_score(product_name, category, price):
    # 1) 先把商品信息拼成一段提示词，
    #    让模型明确知道要分析什么内容
    prompt = f"""你是跨境电商市场风险分析专家。
请分析以下商品的市场风险，并返回严格 JSON 格式：
```

```
商品：{product_name}
品类：{category}
售价：{price}

请评估以下风险维度并返回 JSON：
{{
  "seasonal_risk": 0-10,
  "patent_risk": 0-10,
  "return_risk": 0-10,
  "policy_risk": 0-10,
  "total_risk_score": 0-100,
  "risk_summary": "一句话风险总结"
}}

只返回 JSON，不要任何解释。"""

    # 2) 从环境变量中读取 DashScope API Key
    #    这样比把 Key 直接写死在代码里更安全
    api_key = os.getenv("DASHSCOPE_API_KEY")
    if not api_key:
        raise RuntimeError("Missing environment variable: DASHSCOPE_API_KEY")

    # 3) 调用 DashScope 的兼容 OpenAI 接口
    #    这里用 requests.post 发起 HTTP 请求
    resp = requests.post(
        "https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json"
        },
        json={
            "model": "qwen-plus",
            "messages": [
                {"role": "user", "content": prompt}
            ],
            "temperature": 0.3
        },
        timeout=120
    )

    # 4) 如果 HTTP 请求本身失败，这里会直接抛异常
    resp.raise_for_status()

    # 5) 解析接口返回的 JSON
    result = resp.json()

    # 6) 取出模型真正返回的文本内容
    #    DashScope / OpenAI 兼容格式一般在 choices[0].message.content 里
    if "choices" not in result or not result["choices"]:
        raise RuntimeError(f"Unexpected API response: {result}")

    text = result["choices"][0]["message"]["content"].strip()

    # 7) 尝试把模型返回内容转成 JSON
    #    如果模型没有严格按要求输出 JSON，这里会报错
    try:
        risk_data = json.loads(text)
```

```
    except json.JSONDecodeError:
        # 有些模型会额外包一层 ```json ... ```，这里做一次轻量清洗
        cleaned = text.strip()
        if cleaned.startswith("```"):
            cleaned = cleaned.removeprefix("```json").removeprefix("```").removesuffix("```").strip()

        try:
            risk_data = json.loads(cleaned)
        except json.JSONDecodeError:
            raise RuntimeError(f"Model did not return valid JSON: {text}")

    # 8) 检查关键字段是否存在
    if "total_risk_score" not in risk_data:
        raise RuntimeError(f"Missing total_risk_score in result: {risk_data}")

    # 9) 把“风险分”换算成“安全分”
    #    风险越高，安全分越低
    safety_score = 100 - float(risk_data["total_risk_score"])

    # 10) 返回安全分和完整风险结构
    return round(safety_score, 1), risk_data


def main():
    # 1) 检查命令行参数数量
    #    这里期望至少有 3 个参数：
    #    商品、品类、售价
    if len(sys.argv) < 4:
        print(json.dumps({
            "success": False,
            "error": "Usage: python scoring_engine.py <商品> <品类> <售价>"
        }, ensure_ascii=True))
        sys.exit(1)

    # 2) 从命令行参数中读取输入值
    #    sys.argv[0] 是脚本名，所以真正的参数从 sys.argv[1] 开始
    product_name = sys.argv[1]
    category = sys.argv[2]
    price = sys.argv[3]

    try:
        # 3) 调用核心函数，拿到安全分和风险明细
        safety_score, risk_data = get_risk_score(product_name, category, price)

        # 4) 组织输出结果
        #    这里的输出会被 n8n Execute Command 节点抓取到 stdout
        output = {
            "success": True,
            "product_name": product_name,
            "category": category,
            "price": price,
            "safety_score": safety_score,
            "risk_data": risk_data
        }

        # 5) 打印 JSON 到标准输出
        print(json.dumps(output, ensure_ascii=True))
```

```
        sys.exit(0)

    except Exception as e:
        # 6) 如果执行过程中出错，也输出 JSON 格式错误信息
        #    这样 n8n 后面节点可以更容易判断问题
        error_output = {
            "success": False,
            "error": str(e),
            "product_name": product_name,
            "category": category,
            "price": price
        }

        # 7) stdout 保留给 n8n 读取，stderr 也补一份，方便排障
        print(json.dumps(error_output, ensure_ascii=True))
        print(json.dumps(error_output, ensure_ascii=True), file=sys.stderr)
        sys.exit(1)


# 7) Python 脚本入口
#    当你用命令行执行这个文件时，会从这里开始跑
if __name__ == "__main__":
    main()
```

这里最容易出错的地方其实不是 Python，而是模型返回的格式。因为后面要用 json.loads(text) 解析，所以模型返回的内容必须是严格 JSON，不能夹杂解释文字、代码块标记或者多余符号。

怎么理解这个结果？例如 total_risk_score = 35，说明风险不算高，那么 safety_score = 65；如果 total_risk_score = 80，说明风险偏高，那么 safety_score 只剩 20。这样一来，总分就能继续保持“越高越好”的统一方向。

测试：

1）Set 节点：构造输入参数

Fields to Set 配置：

（1）商品：amanson ASIN:B0GL6J4W2S,Automatic Cat Litter Box Self Cleaning - App Control, Cat Toilet Records, Suitable for Big Cats, Medium Cats, Kittens and Multi-Cat Families, Anti-Pinch, Low Noise【Assembly Required】- White

（2）品类：FRCFRC

（3）售价：129

2）Execute Command 节点：调用 Python 风险评分函数

（1）Execute Once：打开

（2）Command 配置：

python d:\n8n-scripts\scoring_engine.py "{{ $json['商品'] }}" "{{ $json['品类'] }}" "{{ $json['售价'] }}"

3）Edit Fields 节点：risk_summary

Fields to Set 配置：

risk_summary：{{ JSON.parse($json.stdout).risk_data.risk_summary }}

预期输出：

[{

"risk_summary": "高专利侵权风险（同类产品密集且核心清洁机制易涉 Litter Robot 等专利），中高退货率（自动猫砂盆普遍存在误触发、结块兼容性及 APP 连接稳定性问题），中等政策风险（含电机/电子组件，面临 UL/FCC 合规及亚马逊电器类目审核趋严）"

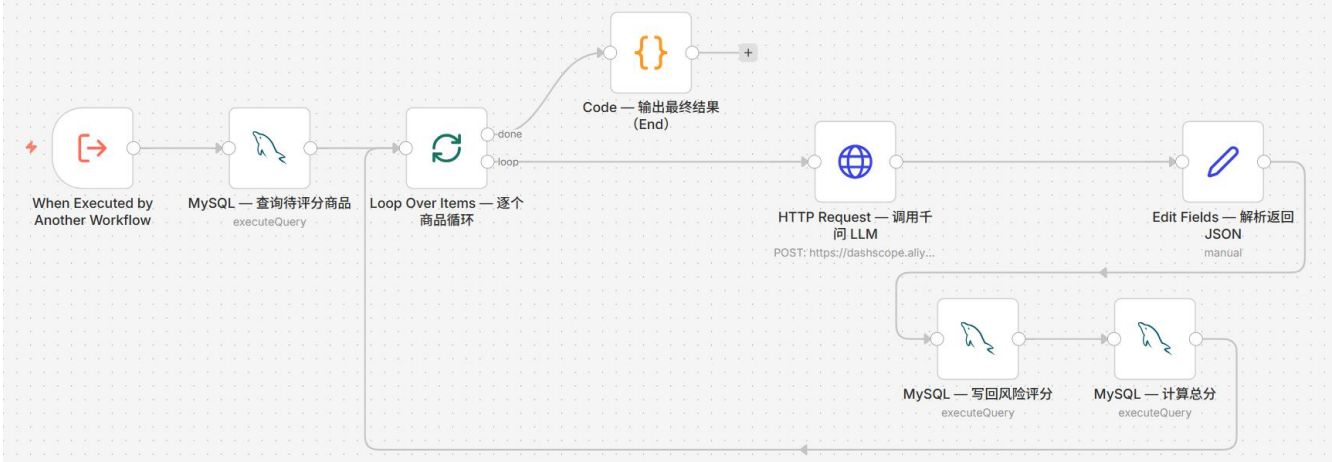
}]

无代码方案：用 N8N 节点实现 LLM 风险评分（推荐零基础学员）

> **适用人群**
> 没有 Python 基础的学员可以跳过上面的 Python 实现，直接使用本节的 N8N 工作流。两种方案功能完全等价，最终都会把 risk_score 和 risk_summary 写入 product_scores 表。N8N 方案学到的是更通用的「工作流编排能力」，正是当下 AI Agent 开发的核心范式。

工作流总览（02：sub_risk_score —— LLM 市场风险评分，共 5 个节点）

```
[Manual Trigger]
       ↓
[节点 1: MySQL] 查待评分商品 (risk_score IS NULL)
       ↓
[节点 2: Loop Over Items] 每个商品循环
       ↓
[节点 3: HTTP Request] 调用千问 qwen-plus API
       ↓
[节点 4: Edit Fields] 解析返回 JSON
       ↓
[节点 5: MySQL] UPDATE 回写风险分
       ↓
[节点 6: Code] 输出最终结果（End）
```



节点 1：MySQL — 查询待评分商品

节点类型：MySQL（Execute Query 操作）。查询语句：

```
SELECT asin, product_name, brand, category, price
FROM product_scores
WHERE risk_score IS NULL
LIMIT 20;
```

这一步选出还没有风险分的商品（risk_score 为 NULL 表示未评分）。LIMIT 20 是为了避免一次跑太多商品产生大量 API 调用，建议分批跑。

> 📌 **product_scores 表从哪来**
>
> 本节读取的 product_scores 由前面的 sub_rule_score 子工作流产出——其 rule_score_80 已写好、risk_score 为 NULL，正等本节回填。若你用的是 Python/SQL 客户端方案，则 product_scores 由 5.3.5 的建表 SQL 产生。
>
> 无论哪条路径，请确保先跑完 sub_rule_score（或执行 5.3.5 建表）再运行本工作流，否则节点 1 会查不到 product_scores 表。

节点 2：Loop Over Items — 逐个商品循环

节点类型：Loop Over Items（无需配置参数）。该节点会把节点 1 返回的 20 条记录拆成 20 次循环，每次循环把一条记录传给后续节点。这是 N8N 实现"批量处理"的标准方式，对应 Python 中的 for 循环。

节点 3：HTTP Request — 调用千问 LLM

节点类型：HTTP Request。完整配置如下：

Method：POST

URL：https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions

Authentication：Generic Credential Type → Bear Auth

新建凭证：Bear Auth account

Bearer Token：YOUR_DASHSCOPE_API_KEY

Send Body：开启（On）

Body Content Type：JSON

Specify Body：Using JSON

JSON Body 填写：

```
{
  "model": "qwen3.7-max",
  "messages": [
    {
      "role": "user",
      "content": "=你是跨境电商市场风险分析专家。请分析以下商品的市场风险，返回严格 JSON 格式(注意：risk_summary 字段中的内容请翻译成中文)：\n 商品：{{ $json.product_name }}\n 品类：{{ $json.category }}\n 售价：${{ $json.price }}\n\n 请评估以下风险维度并返回 JSON：\n{\n  \"seasonal_risk\": 0-10,\n  \"patent_risk\": 0-10,\n  \"return_risk\": 0-10,\n  \"policy_risk\": 0-10,\n  \"total_risk_score\": 0-100,\n  \"risk_summary\": \"一句话风险总结\"\n}\n 仅返回 JSON，不要任何解释。"
    }
  ],
  "temperature": 0.3,
```

```
  "response_format": {"type": "json_object"}
}
```

> **关键技巧**
> content 字段值以 = 号开头，表示这是一个 N8N 表达式而不是普通字符串。{{ $json.product_name }} 会被自动替换为当前循环商品的名称。response_format 强制模型返回 JSON，避免解析失败。

节点 4：Edit Fields — 解析返回 JSON

节点类型：Edit Fields（之前叫 Set 节点）。Mode 选 Manual Mapping，添加以下 3 个字段（全部用「表达式」模式，不是固定值）：

字段 1 — asin：{{ $('Loop Over Items — 逐个商品循环').item.json.asin }}

字段 2 — risk_score：{{ Math.round(JSON.parse($('HTTP Request — 调用千问 LLM').first().json.choices[0].message.content).total_risk_score) }}

字段 3 — risk_summary：{{ JSON.parse($('HTTP Request — 调用千问 LLM').first().json.choices[0].message.content).risk_summary }}

字段 2 的表达式做了两件事：

① 把"风险分"转成"安全分"可以在这里做了，后面就不动了，或者后面 sql 计算的时候再减）；

② ②保留一位小数。等价于 Python 中的 round(100 - risk_data["total_risk_score"], 1)。

节点 5：MySQL — 写回风险评分

节点类型：MySQL（Execute Query 操作）。Query：

```
UPDATE product_scores
SET
  risk_score = {{ $json.risk_score }},
  risk_summary = '{{ $json.risk_summary }}'
    WHERE asin = '{{ $json.asin }}';
```

> **⚠ 注意**
> risk_summary 是字符串，必须用单引号包裹。如果 LLM 返回的总结里包含单引号，会导致 SQL 报错，可以在节点 4 加一个字段：cleaned_summary = {{ $json.risk_summary.replace(/'/g, "\\'") }} 做转义。

节点 6：Code — 输出最终结果（End）

节点类型：Code

Mode： Run Once for All Items

Language：JavaScript

Javascript 代码

```
const items = $input.all();

return [
  {
    json: {
      success: items.every(item => item.json.success === true),
      count: items.length,
    }
  }
];
```

教学价值对比

Python 方案教的是「写脚本」，N8N 方案教的是「编排工作流 + 调用 API + 解析 JSON + 写 SQL」——后者正是 AI 应用工程师/AI 产品经理岗位的核心技能。Dify、Coze、Make 等主流 AI 工作流平台底层逻辑完全一致，掌握 N8N 等于掌握了整个工作流编排范式。

5.3.3 写回风险分并计算总分

为了让方向统一，我们把前三维算出来的规则分记为 rule_score_80，也就是满分 80 的规则部分；风险分使用刚才转出来的 safety_score，满分 20。最后相加，刚好是 100 分。

当 LLM 把每个商品的安全分返回以后，就可以把它写回 product_scores。此时总分的计算非常简单：前三维规则分 + 风险安全分 × 20%。

在 02：sub_risk_score —— LLM 市场风险评分工作流中再加一个节点叫：MySQL — 计算总分，节点类型：MySQL（Execute Query 操作）。Query：

```
/* ============================================================
   等 LLM 风险分回来以后，再统一计算最终总分
   注意：这里的 (100-risk_score) 实际存的是“安全分”
   ============================================================ */
UPDATE product_scores
SET total_score = ROUND(
    rule_score_80 + (100-risk_score) * 0.20,
    1
);
```

这里的解读很直观：如果一个商品前三维不错，但风险很高，最终总分还是会被拉下来；反过来，如果前三维一般，但风险很低，也能保住一部分分数。这样就实现了“机会”和“风险”同时被考虑。

> **无代码衔接**
> 本步的 UPDATE product_scores SET total_score 也可以作为一个独立 MySQL 节点，接在 5.3 风险评分工作流之后，构成实验五在 N8N 内的最后一环：sub_rule_score → 风险评分 → 本合成节点。三段串起来，实验五即可全程在 N8N 内一键跑通。

5.3.4 四维加权合成 + 排名

```
/* 查看最终的 TOP20 候选商品 */
SELECT
    asin,
    product_name,
    price,
    trend_score,
    competition_score,
    profit_score,
    risk_score,
    total_score
FROM product_scores
ORDER BY total_score DESC
LIMIT 20;
```

最终的 TOP20 不是“绝对正确答案”，而是“当前数据下更值得优先看的候选清单”。在教学里，这个结果最重要的价值不是排名本身，而是让我们理解：四个维度是如何一步一步合成一个可解释的选品分数的。

到这里，四维评分的教学链路就完整打通了：先拼底表，再算前三维，再接 LLM 风险分，最后合成总分。这样写出来的 SQL 不但能跑，而且每一步都能单独讲、单独看、单独验证。

实验六：Vision 多模态商品图片分析 ★

ƒ **时长：**90 分钟　ƒ **目标：**让 AI “看懂”商品图片，提取视觉特征用于选品决策

ƒ **前置条件：**通义千问 DashScope API Key（支持 qwen-vl-max 模型）、N8N 已部署

6.1 为什么需要“看”图片？

文字数据有盲区。两个猫砂盆价格和评分一样，但一个是北欧极简风、一个是廉价塑料感——图片决定了转化率。Vision 多模态分析能提取人眼看到但文字无法表达的信息。

6.2 通义千问 VL API 环境准备

本实验使用通义千问 VL（qwen-vl-max）作为 Vision 模型，它是国内访问最稳定、成本最低的多模态大模型之一。

获取 DashScope API Key

1. **访问阿里云 。继续使用我们之前的 key**
2. **登录阿里云账号，**如果没有则免费注册
3. **进入“API-KEY 管理”，**点击“创建 API-KEY”
4. **复制 API Key，**保存好（后面所有代码中的 YOUR_DASHSCOPE_API_KEY 都替换为这个值）

为什么选通义千问 VL 而不是 GPT-4o?

① 国内访问稳定，不需要翻墙；② DashScope API 兼容 OpenAI 格式，迁移成本几乎为零；③ 千问 VL-Max 的视觉理解能力在国内模型中属于第一梯队；④ 新用户有免费额度，教学环境够用。

快速验证：Python 调用测试

> D:\n8n-scripts\vision_analyzer.py

```
 import os
import sys
import json
import requests

# 尽量统一 UTF-8 输出，减少 Windows / n8n 环境下的乱码问题
try:
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
except Exception:
    pass
```

```
def analyze_product_image(image_url):
    # 1) 从环境变量读取 DashScope API Key
    api_key = os.getenv("DASHSCOPE_API_KEY")
    if not api_key:
        raise RuntimeError("Missing environment variable: DASHSCOPE_API_KEY")

    # 2) 将提示词直接写死在代码中
    prompt_text = """你是一位跨境电商视觉分析专家，专注于 Amazon 商品图片分析。
请分析这张商品主图，从电商转化率角度评估，返回严格 JSON（注意：返回文字尽量用中文）：

{
    "dominant_colors": ["主色 1", "主色 2"],
    "style_tags": ["标签"],
    "target_audience": "目标人群一句话描述",
    "visual_quality_score": 【0-10 浮点数，保留 1 位小数】,
    "selling_point_from_image": "从图片中能看出的核心卖点",
    "competitor_similarity": "与哪类竞品视觉风格相似"
}

style_tags 从以下选择：
极简/复古/可爱/科技/奢华/户外/北欧/日系/美式/工业

评分标准：
- 1-3 分：图片模糊、背景杂乱、产品不突出
- 4-6 分：图片清晰但无特色
- 7-8 分：构图专业、风格明确
- 9-10 分：视觉冲击力强、能直接促成购买

硬性规则： 1. visual_quality_score 根据图片真实质量独立评估，严禁持续输出相同分值； 2. style_tags 只能使用给定标签，不能自创； 3. 仅返回纯净 JSON，不要任何解释、前言、markdown 标记。"""

    # 3) 调用 DashScope 兼容 OpenAI 的多模态接口
    resp = requests.post(
        "https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json"
        },
        json={
            "model": "qwen-vl-max",
            "messages": [
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "image_url",
                            "image_url": {
                                "url": image_url
                            }
                        },
                        {
                            "type": "text",
```

```
                            "text": prompt_text
                        }
                    ]
                }
            ],
            "temperature": 0.2
        },
        timeout=120
    )

    # 4) HTTP 出错直接抛异常
    resp.raise_for_status()

    # 5) 解析返回结果
    result = resp.json()

    # 6) 取出模型输出内容
    if "choices" not in result or not result["choices"]:
        raise RuntimeError(f"Unexpected API response: {result}")

    content = result["choices"][0]["message"]["content"].strip()

    # 7) 尝试把模型输出解析为 JSON
    try:
        parsed = json.loads(content)
    except json.JSONDecodeError:
        cleaned = content.strip()
        if cleaned.startswith("```"):
            cleaned                                                                    =
cleaned.removeprefix("```json").removeprefix("```").removesuffix("```").strip()
        parsed = json.loads(cleaned)

    return parsed, result


def main():
    # 1) n8n 通过命令行传入图片 URL
    #    argv[1] = image_url
    if len(sys.argv) < 2:
        error_output = {
            "success": False,
            "error": "Usage: python script.py <image_url>"
        }
        print(json.dumps(error_output, ensure_ascii=True))
        sys.exit(1)

    image_url = sys.argv[1]

    try:
        # 2) 调用视觉分析接口
        parsed, raw_result = analyze_product_image(image_url)

        # 3) 输出给 n8n 直接接收
        output = {
            "success": True,
            "image_url": image_url,
            "analysis": parsed,
            "raw_result": raw_result
```

```
        }

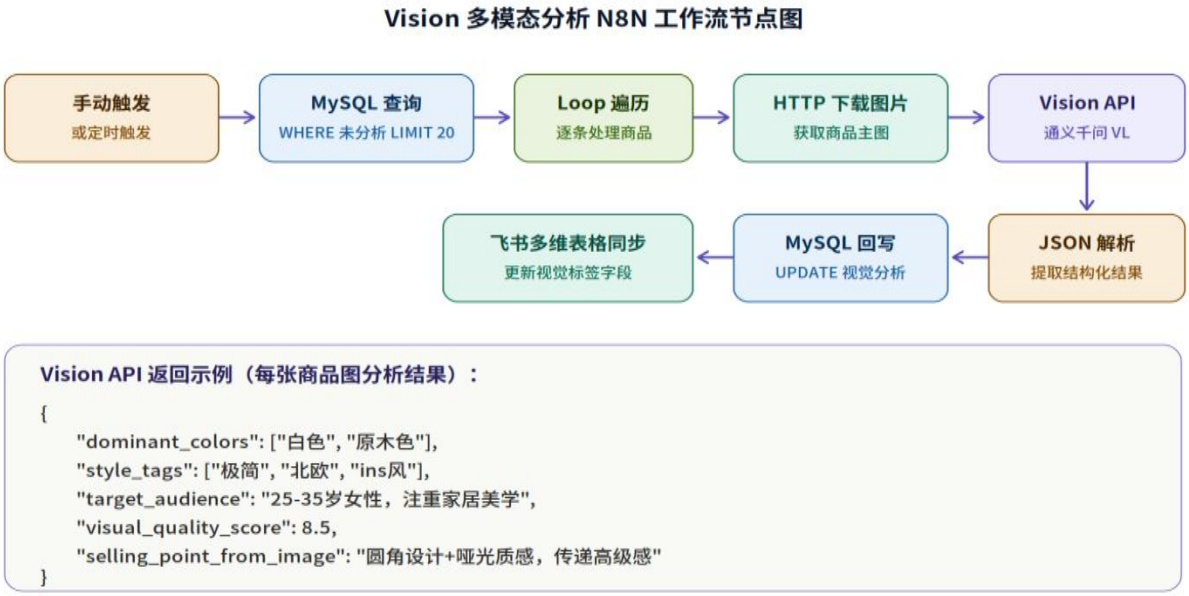
        print(json.dumps(output, ensure_ascii=True))
        sys.exit(0)

    except Exception as e:
        # 4) 出错时也输出结构化 JSON，便于 n8n 排查
        error_output = {
            "success": False,
            "error": str(e),
            "image_url": image_url
        }
        print(json.dumps(error_output, ensure_ascii=True))
        print(json.dumps(error_output, ensure_ascii=True), file=sys.stderr)
        sys.exit(1)


if __name__ == "__main__":
    main()
```

6.3 Prompt 工程迭代：从废话到精准

这是本实验的核心教学环节。我们会用同一张商品图片，体验三版 Prompt 的效果差异：



Vision API 返回示例（每张商品图分析结果）：

```
{
    "dominant_colors": ["白色", "原木色"],
    "style_tags": ["极简", "北欧", "ins风"],
    "target_audience": "25-35岁女性，注重家居美学",
    "visual_quality_score": 8.5,
    "selling_point_from_image": "圆角设计+哑光质感，传递高级感"
}
```

V1 版 Prompt：简单提问（反面教材）

```
描述这张图片
```

返回结果："这是一个白色的设备……" —— 一堆废话，无法结构化提取。

V2 版 Prompt：结构化指令

```
请分析这张商品图片，返回 JSON 格式：
{
```

```
    "dominant_colors": ["主色 1", "主色 2"],
    "style_tags": ["标签 1", "标签 2"],
    "target_audience": "目标人群描述",
    "visual_quality_score": 7.5,
    "selling_point_from_image": "卖点描述"
}
仅返回 JSON，不要任何解释。
```

返回结果：JSON 格式正确了，但分析深度不够，“卖点”描述太笼统。

V3 版 Prompt：角色 + 约束 + 示例（最终版）

```
你是一位跨境电商视觉分析专家，专注于 Amazon 商品图片分析。
请分析这张商品主图，从电商转化率角度评估，返回严格 JSON：

{
    "dominant_colors": ["主色 1", "主色 2"],
    "style_tags": ["标签"],
    "target_audience": "目标人群一句话描述",
    "visual_quality_score": 【0-10 浮点数，保留 1 位小数】,
    "selling_point_from_image": "从图片中能看出的核心卖点",
    "competitor_similarity": "与哪类竞品视觉风格相似"
}

style_tags 从以下选择：
极简/复古/可爱/科技/奢华/户外/北欧/日系/美式/工业

评分标准：
- 1-3 分：图片模糊、背景杂乱、产品不突出
- 4-6 分：图片清晰但无特色
- 7-8 分：构图专业、风格明确
- 9-10 分：视觉冲击力强、能直接促成购买

硬性规则： 1. visual_quality_score 根据图片真实质量独立评估，严禁持续输出相同分值； 2. style_tags 只能使用给定标签，不能自创； 3. 仅返回纯净 JSON，不要任何解释、前言、markdown 标记。
```

教学重点

Prompt 工程不是一步到位的。V1→V2→V3 的迭代过程本身就是最重要的教学内容：“写→测→改→测”循环。课堂上建议让学员自己试三版的效果差异。

6.4 N8N 工作流实现批量 Vision 分析

> 具体见子工作流：sub_vision_analyzer —— Vision 多模态 N8N 工作流

用 N8N 把 Vision 分析串成自动化工作流，批量处理 20 款商品图片。

教学说明

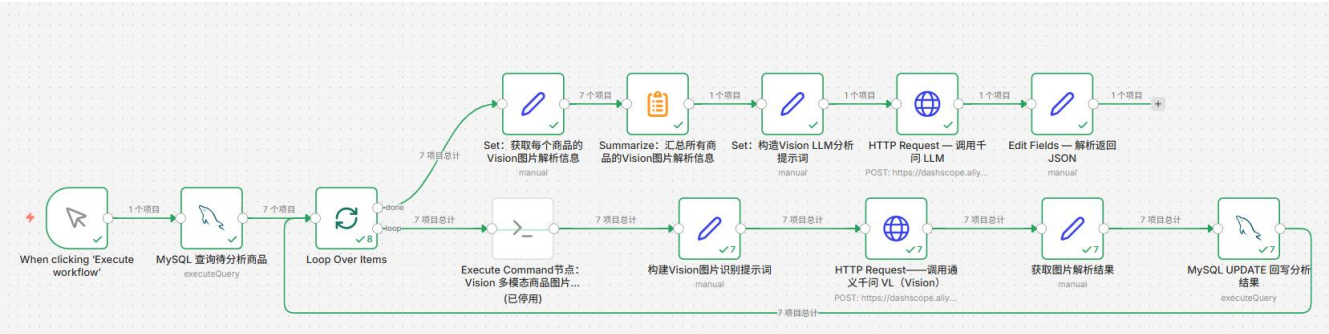
「分数 + 解释」配套存储是企业级 AI 评分系统的标配。vision_score 是数字（用于排序、过滤），vision_data 是结构化 JSON（用于展示原因）。LLM/Vision 调用按 token 收费，把推理过程一起落库，可以做到「一次调用，长期复用」，这是企业落地大模型的核心成本控制原则。

✅本前置准备交付物

raw_products 表新增 vision_analyzed、vision_score、vision_data 三个字段，DESC 命令能查询到，所有商品 vision_analyzed 默认为 0。

版本一、Python 版

```
[Manual Trigger]
        ↓
[节点 1: MySQL] MySQL 查询待分析商品（vision_analyzed = 0）
        ↓
[节点 2: Loop Over Items] 每个商品循环
        ↓
[节点 3: Execute Command] Vision 多模态商品图片分析，调用千问 qwen-plus API
或
[节点 3: Edit Fields] 构建 Vision 图片识别提示词
        +
[节点 4: HTTP Request] HTTP Request—调用通义千问 VL（Vision）
        ↓
[节点 5: Edit Fields] 获取图片解析结果
        ↓
[节点 6: MySQL] MySQL UPDATE 回写分析结果
        ↓
[节点 7: Edit Fields] 获取每个商品的 Vision 图片解析信息
        ↓
[节点 8: Summarize] 汇总所有商品的 Vision 图片解析信息
        ↓
[节点 9:  Edit Fields] Set：构造 Vision LLM 分析提示词
        ↓
[节点 10:  HTTP Request] 调用千问 LLM
        ↓
[节点 11:  Edit Fields] 解析返回 JSON
```



节点 1：MySQL 查询待分析商品

```
节点类型：MySQL
查询：
     SELECT asin, title, image_url
    FROM raw_products
    WHERE vision_analyzed = 0
      AND crawl_date = CURDATE()    -- ← 加这行，只分析今天新采的商品
    LIMIT 20;
```

节点 2：Loop Over Items

```
节点类型：Loop Over Items
每个商品循环
```

节点 3：Execute Command 节点：Vision 多模态商品图片分析 (或 http 形式执行 python)

此处跳过

Command：

```
python d:\n8n-scripts\vision_analyzer.py "{{ $json.image_url }}"
```

节点 4：Set：构建 Vision 图片识别提示词

```
prompt：

你是一位跨境电商视觉分析专家，专注于 Amazon 商品图片分析。 请分析这张商品主图，从电商转化率角度评估，返回严格 JSON：
{
"dominant_colors": ["主色 1", "主色 2"],
"style_tags": ["标签"],
"target_audience": "目标人群一句话描述",
"visual_quality_score": 【0-10 浮点数，保留 1 位小数】,
"selling_point_from_image": "从图片中能看出的核心卖点",
"competitor_similarity": "与哪类竞品视觉风格相似"
}
style_tags 从以下选择： 极简/复古/可爱/科技/奢华/户外/北欧/日系/美式/工业
评分标准：
- 1-3 分：图片模糊、背景杂乱、产品不突出
- 4-6 分：图片清晰但无特色
- 7-8 分：构图专业、风格明确
- 9-10 分：视觉冲击力强、能直接促成购买

硬性规则： 1. visual_quality_score 根据图片真实质量独立评估，严禁持续输出相同分值； 2. style_tags 只能使用给定标签，不能自创； 3. 仅返回纯净 JSON，不要任何解释、前言、markdown 标记。
```

节点 5：HTTP Request——调用通义千问 VL（Vision）

节点类型：HTTP Request。完整配置如下：

Method：POST

URL：https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions

Authentication：Generic Credential Type → Bear Auth

新建凭证：Bear Auth account

Bearer Token：YOUR_DASHSCOPE_API_KEY

Send Body：开启（On）

Body Content Type：JSON

Specify Body：Using JSON

JSON Body 填写：

```
{
    "model": "qwen-vl-max",
    "messages": [{
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": "{{ $('Loop Over Items').item.json.image_url }}"
                }
            },
            {
                "type": "text",
                "text": {{ JSON.stringify($json.prompt) }}
            }
        ]
    }],
    "temperature": 0.3,
    "response_format": {"type": "json_object"}
}
```

说明：为防止下载图片失败，建议最好在这个 Http Request 节点的设置中，设置失败时重试，最大尝试次数设置为 3，等待尝试之间的时间（毫秒）设置为 1000.

节点 6：Set：获取图片解析结果

```
Fields to Set:

vision_data: {{ $json.choices[0].message.content }}
vision_colors: {{ JSON.parse($json.choices[0].message.content).dominant_colors}}
vision_style: {{ JSON.parse($json.choices[0].message.content).style_tags}}
vision_audience: {{ JSON.parse($json.choices[0].message.content).target_audience }}
vision_score: {{ JSON.parse($json.choices[0].message.content).visual_quality_score }}
vision_selling_pt:
{{ JSON.parse($json.choices[0].message.content).selling_point_from_image }}
competitor_similarity:
{{JSON.parse($json.choices[0].message.content).competitor_similarity }}
```

节点 7：MySQL UPDATE 回写分析结果

```
Operation: Execute SQL
```

```
Query:
UPDATE raw_products SET
    vision_data       = '{{ $json.vision_data }}',
    vision_colors     = '{{$json.vision_colors  }}',
    vision_style      = '{{$json.vision_style }}',
    vision_audience   = '{{ $json.vision_audience }}',
    vision_score      =  {{ $json.vision_score }},
    vision_selling_pt = '{{$json.vision_selling_pt }}',
    vision_analyzed   = 1
WHERE asin = '{{ $('节点 2: Loop Over Items').item.json.asin }}'
  AND crawl_date = CURDATE();    --    -- ← 加这一行
```

6.5 视觉竞争力矩阵分析（本实验高潮）

把 7 款商品的 Vision 分析结果汇总，用 LLM 做视觉竞争力分析：

节点 8：Set：获取每个商品的 Vision 图片解析信息

在 Loop Over Items 节点后面接入 Edit Fields 节点：

Fields to Set：

vision_info:

```
{{ $itemIndex + 1 + '、' +  $('节点 1: MySQL 查询待分析商品').first().json.asin +'|'
+$('节点 1: MySQL 查询待分析商品').first().json.title + '|' + '视觉主色调: '+  $('节点
1: MySQL 查询待分析商品').first().json.vision_colors+ '|'+ '视觉风格: '+  $('节点 1:
MySQL 查询待分析商品').first().json.vision_style +'|'+'目标人群: ' +  $('节点 1: MySQL
查询待分析商品').first().json.vision_audience + '|'+'视觉评分: '+  $('节点 1: MySQL 查
询待分析商品').first().json.vision_score + '|'+'核心卖点: '+  $('节点 1: MySQL 查询待分
析商品').first().json.vision_selling_pt }}
```

节点 9：Summarize：汇总所有商品的 Vision 图片解析信息

```
Aggregation: Concatendate
Field: vision_info
Separator: New Line
```

节点 10：Set：构造 Vision LLM 分析提示词

Fields to Set:

prompt:

以下是 三个```中的内容是 7 款智能猫砂盆的视觉特征分析数据：

```
{{ $json.concatenated_vision_info }}
```

请从跨境电商视觉营销角度回答：
1. 哪些产品的视觉风格高度同质化？（红海警告）
2. 存在哪些视觉空白区？（蓝海机会）
3. 新品主图应该主打什么风格才能差异化？
请严格按以下 JSON 格式返回，不要输出任何解释：

```
{
  "red_ocean_products": [
    {
```

```
      "asin": "B0XXXXXXX",
      "product_name": "产品名",
      "visual_style": ["科技", "极简"],
      "similarity_reason": "与哪些竞品视觉高度相似"
    }
  ],
  "blue_ocean_opportunities": [
    {
      "opportunity": "视觉空白区描述",
      "why_it_matters": "为什么值得切入",
      "recommended_style": ["北欧", "日系"]
    }
  ],
  "differentiation_strategy": {
    "main_style": "建议主打风格",
    "main_color_direction": ["主色 1", "主色 2"],
    "main_visual_message": "主图应传达的核心感受"
  },
  "summary": "一句话结论",
  "concise_report": "请给出回答以上三个问题的视觉竞争力矩阵分析简报，要给老板看的"
}
```

节点 11：HTTP Request — 调用千问 LLM

节点类型：HTTP Request。完整配置如下：

```
Method：POST
URL：https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions
Authentication：Generic Credential Type → Bear Auth
新建凭证：Bear Auth account
Bearer Token：YOUR_DASHSCOPE_API_KEY
Send Body：开启（On）
Body Content Type：JSON
Specify Body：Using JSON
JSON Body 填写：
{
  "model": "qwen-plus",
  "messages": [
    {
      "role": "user",
      "content": {{ JSON.stringify($json.prompt) }}
    }
  ],
  "temperature": 0.3,
  "response_format": {"type": "json_object"}
}
```

节点 12：Edit Fields — 解析返回 JSON

```
Fields to Set:

红海警告:
{{ JSON.parse($json.choices[0].message.content).red_ocean_products }}

蓝海机会:
{{ JSON.parse($json.choices[0].message.content).blue_ocean_opportunities }}
```

```
新品主图风格建议：
{{ JSON.stringify(JSON.parse($json.choices[0].message.content).differentiation_strat
egy, null, 2) }}

一句话建议：
{{ JSON.parse($json.choices[0].message.content).summary }}

视觉竞争力矩阵分析简报：
{{ JSON.parse($json.choices[0].message.content).concise_report }}
```

最终预期出：

一句话建议：
六款产品陷入‘科技极简白灰墙’视觉内卷，唯一破局点在于用北欧/日系生活化美学重构智能猫砂盆的情感价值。

视觉竞争力矩阵分析简报：
视觉竞争力矩阵：红海=6/7 款同质化科技极简（白/灰/黑主色+纯色硬照）；蓝海=生活场景沉浸（北欧/日系）+自然暖色系（燕麦米/灰绿）+有机材质表达；建议新品主图放弃参数轰炸，主打‘晨光中的静音猫砂盆+猫咪慵懒入镜’生活切片，以温度破技术壁垒。

✓本实验交付物

- N8N Vision 工作流搭建完成
- 7 款商品视觉分析数据
- 视觉竞争力矩阵分析报告
- V3 版 Vision Prompt 模板

实验七：竞品深度解剖——差评挖掘与定价策略

时长：60 分钟　**目标：**找到竞争对手的弱点，确定差异化策略

7.1 竞品分析的核心逻辑

竞品分析不是“谁便宜”，而是“敌人的弱点在哪”。差评里藏着用户最真实的痛点，这些痛点直接变成你的卖点。



7 款商品的角色划分（重要前置说明）

本课程数据集里的 7 款商品分为两个角色，本节差评挖掘的对象是其中 4 款竞品：

【3 款候选商品】自家可能要卖的（不参与差评分析）

- B0DZSY869Z — Mintakawa Automatic Cat Litter Box
- B0DK6RSFG2 — Neakasa M1 Plus Lite
- B0GL6J4W2S — FRCFRC（白牌入门款）

【4 款竞品】市场上已经在卖的对手（差评分析对象）

- B0FFDYB7K8 — Litter-Robot 4（高端旗舰，价格带 $800+）
- B0DFYF2D7D — PETKIT PuraMax 2（中高端，价格带 $299）
- B0CGRQTH9P — CATLINK Scooper SE（中端，价格带 $319）
- B07WZPJ2LW — PetSafe ScoopFree（成熟老牌，多档可选）

7.3 差评挖掘的对象就是这 4 款竞品——只有“敌人”才有让我们挖弱点的价值。自家 3 款候选商品还没卖出去、没有真实差评可挖，也不需要做这一步。下一节 SQL 里的 4 个 ASIN 占位符，依次对应上面这 4 款。

7.2 评论数据从哪来？——企业级获取渠道详解

你可能已经注意到，Amazon 页面上只显示 8-10 条评论，而且翻页受限。这是 Amazon 从 2025 年开始的反爬策略。但企业实战中，评论数据通过以下渠道获取：

渠道一：Amazon SP-API（官方 API）

- **网址**：https://developer-docs.amazon.com/sp-api/
- **费用**：免费，但需要有 Amazon 卖家账号（Professional 账号 $39.99/月）
- **能获取什么**：Customer Feedback API 提供评论主题分析、正负面趋势、星级分布等结构化数据
- **限制**：只能查看自己店铺的评论，不能直接查看竞品评论原文。但新增的 Feedback Insights 接口可获取品类级别的评论主题分析
- **面试话术**："我们通过 Amazon SP-API 的 Customer Feedback API 获取评论主题分析数据，这是官方提供的合规渠道"

渠道二：Helium 10（行业标杆工具）

- **网址**：https://www.helium10.com/
- **费用**：Starter $39/月（年付）/ $49/月，Platinum $79/月（年付）/ $129/月，Diamond $229/月（年付）/ $359/月
- **评论相关功能**：Review Insights 功能可查看任意竞品的评论分析（需要 Platinum 及以上），提供评论情感分析、关键词提取、趋势变化
- **是否需要卖家账号**：不需要。普通 Amazon 买家账号即可注册使用
- **免费版**：有永久免费版，但功能受限（每天只能用少量次数），不需要绑信用卡即可注册
- **面试话术**："评论数据通过 Helium 10 Review Insights 获取，这是行业最广泛使用的竞品分析工具，公司购买了 Platinum 订阅"

渠道三：Jungle Scout（另一大主流工具）

- **网址**：https://www.junglescout.com/
- **费用**：Starter $49/月，Growth Accelerator $79/月，Brand Owner + CI $399/月（年付有折扣）
- **评论相关功能**：Review Automation + Competitive Intelligence 可分析竞品评论趋势和关键词
- **是否需要卖家账号**：不需要。但要充分使用需要绑定 Seller Central
- **特色**：7 天无条件退款保证，可以先试用再决定

渠道四：专业评论数据 API（按调用量付费）

- **Canopy API**：https://www.canopyapi.co/ —— 免费版 100 次/月，付费版按量计费，高级版 $400/月。支持 REST 和 GraphQL 接口，可获取任意商品的全量评论原文
- **Unwrangle API**：https://docs.unwrangle.com/ —— 按调用次数计费，支持按星级筛选、分页获取、最新评论优先。注意：2025 年起 Amazon 要求提供 Cookie 才能访问评论

课堂数据与面试话术的关系

课堂上我们用预准备的评论数据包做分析。但面试时你应该说：“评论数据通过公司购买的 Helium 10 Platinum 订阅获取，这是行业标准的竞品分析工具」或者「我们通过 Amazon SP-API 的 Customer Feedback API 获取合规数据」——这两种说法都是真实可信的，因为这确实是企业的标准做法。

7.3 LLM 差评批量分析

从 MySQL 提取 4 款竞品的差评数据，用 LLM 批量分析：

```
-- 提取竞品差评
  SELECT
    rp.asin,
    rp.brand,
    pr.review_text,
    pr.rate
  FROM raw_products rp
  INNER JOIN product_reviews pr ON rp.asin = pr.asin
  WHERE pr.rate <= 3 and rp.asin  IN ('竞品 1_ASIN', '竞品 2_ASIN', '竞品 3_ASIN', '竞品 4_ASIN')
  ORDER BY rp.asin DESC;
```

完整 SQL：

```
-- 提取竞品差评
SELECT
  rp.asin,
  rp.brand,
  pr.review_text,
  pr.rate
FROM raw_products rp
INNER JOIN product_reviews pr ON rp.asin = pr.asin
WHERE pr.rate <= 3 and rp.asin IN ('B0FFDYB7K8', 'B0DFYF2D7D', 'B0CGRQTH9P', 'B07WZPJ2LW')
ORDER BY rp.asin DESC;
```

LLM 差评分析 Prompt

```
你是跨境电商竞品分析专家。
以下是 Litter-Robot 4 在 Amazon 上的 20 条差评。

{差评文本}

请分析并返回 JSON：
{
    "top5_pain_points": [
        {
            "keyword": "漏尿",
            "count": 8,
            "sample_quote": "一句原文摘要(英文, <15 字)"
        }
    ],
    "differentiation_opportunities": [
        "针对漏尿问题，可以主打密封设计",
```

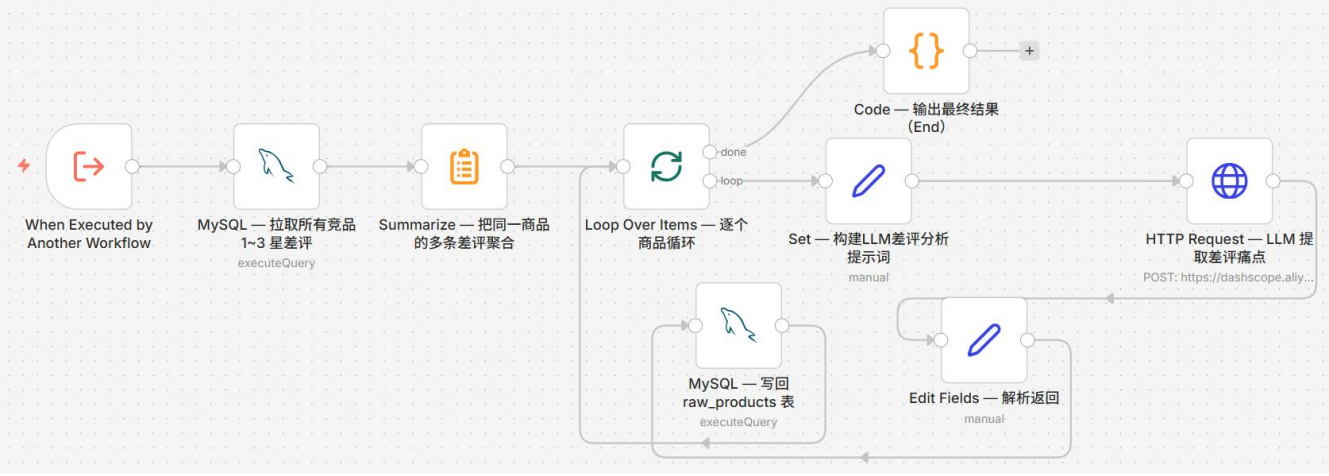
```
        "针对价格问题，可以定价$250-350 区间"
    ]
}
仅返回 JSON。
```

无代码方案：用 N8N 实现差评批量分析

> **本节目标**
> 把上面 Python 实现的「读取差评 → LLM 提取痛点 → 写回数据库」流程，改造成完整可执行的 N8N 工作流。同样的功能，零代码。

工作流总览（04：sub_competitor_analyzer —— 差评批量分析，共 8 个节点）

```
[Manual Trigger]
       ↓
[节点 1: MySQL] 查所有竞品的差评（按 ASIN 聚合）
       ↓
[节点 2: Summarize] 把同一 ASIN 的多条差评聚合成一段文本
       ↓
[节点 3: Loop Over Items] 每个商品循环
       ↓
[节点 4: Loop Over Items] 每个商品循环
       ↓
[节点 5: Edit Fields] 构建 LLM 差评分析提示词
       ↓
[节点 6: Edit Fields] 解析返回的 JSON 痛点列表
       ↓
[节点 7: MySQL] UPDATE 写回 pain_points 字段
       ↓
[节点 8: Code] 输出最终结果（End）
```



节点 1：MySQL — 拉取所有竞品的 1~3 星差评

```
SELECT
  rp.asin,
```

```
  rp.brand,
  pr.review_text,
  pr.rate
FROM raw_products rp
INNER JOIN product_reviews pr ON rp.asin = pr.asin
WHERE pr.rate <= 3
   -- and rp.asin IN ('B0FFDYB7K8', 'B0DFYF2D7D', 'B0CGRQTH9P', 'B07WZPJ2LW')
  AND pr.crawl_date = CURDATE()   -- ← 加这一行
ORDER BY rp.asin DESC;
```

节点 2：Summarize — 把同一商品的多条差评聚合

节点类型：Summarize。配置：

Aggregation：Append

Field：review_text

Fields to Split By：asin,brand

Add option --> Output Format：Each Split in a Separate Item

该节点在 n8n 里完成“按 ASIN 和 Brand 分组 + 收集 review_text”

节点 3：Loop Over Items — 逐个商品循环

节点类型：Loop Over Items（无需配置参数）。把每个商品的差评聚合数据传给下游节点。

节点 4：Set — 构建 LLM 差评分析提示词

节点类型：Edit Fields。把每个商品的差评聚合数据拼接成 LLM 差评分析提示词传给下游节点。

Fields to Set：

all_reviews：

```
"以下是商品的所有差评：\n\n{{$json.appended_review_text.join('\n---\n')}}\n\n 请提取核心痛点，返回严格 JSON：\n{\n  \"top_pain_points\": [\"痛点 1\", \"痛点 2\", \"痛点 3\"],\n \"pain_summary\": \"一句话总结\",\n  \"improvement_suggestions\": [\"建议 1\", \"建议 2\"]\n}\n 仅返回 JSON。"
```

Include Other Input Fields：true

Add option -->Ignore Type Conversion Errors：true

节点 5：HTTP Request — LLM 提取差评痛点

Method：POST

URL：https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions

JSON Body：

```
{
  "model": "qwen-plus",
  "messages": [
    {
      "role": "system",
      "content": "你是亚马逊商品评论分析专家。请只输出 JSON。"
    },
    {
```

```
      "role": "user",
      "content": {{ JSON.stringify($json.all_reviews) }}
    }
  ],
  "temperature": 0.3,
  "response_format": { "type": "json_object" }
}
```

节点 6：Edit Fields — 解析返回

asin：{{ $('Loop Over Items').item.json.asin }}

pain_points：{{ JSON.parse($json.choices[0].message.content).top_pain_points.join(' | ') }}

pain_summary：{{ JSON.parse($json.choices[0].message.content).pain_summary }}

节点 7：MySQL — 写回 raw_products 表

需要在 raw_products 表先 ALTER TABLE 添加两个字段：

```
ALTER TABLE raw_products
ADD COLUMN pain_points TEXT COMMENT '差评核心痛点',
ADD COLUMN pain_summary VARCHAR(500) COMMENT '差评一句话总结';
```

这里如果前面已经执行过了，上面的 SQL 可以不用执行。

然后节点 7 的 Query：

```
UPDATE raw_products
SET
  pain_points = '{{ $json.pain_points }}',
  pain_summary = '{{ $json.pain_summary }}'
WHERE asin = '{{ $json.asin }}'
  AND crawl_date = CURDATE();   -- ← 加这一行
```

> ☑ **验证**
> 执行后查询 SELECT asin, brand, pain_summary FROM raw_products WHERE pain_summary IS NOT NULL LIMIT 5; 能看到每个商品都有了一句话差评总结，说明无代码方案完整跑通了。

节点 8：Code — 输出最终结果（End）

节点类型：Code

Mode： Run Once for All Items

Language：JavaScript

Javascript 代码

```
const items = $input.all();

return [
  {
    json: {
      success: items.every(item => item.json.success === true),
      count: items.length,
    }
  }
];
```

7.4 价格带分析

用 SQL 计算 TOP50 商品的价格分布，找到“价格真空带”：

```
-- 价格带分析
SELECT
    CASE
        WHEN price < 150  THEN '$0-150（低端）'
        WHEN price < 250  THEN '$150-250（中低端）'
        WHEN price < 400  THEN '$250-400（中端）'
        WHEN price < 600  THEN '$400-600（中高端）'
        ELSE '$600+（高端）'
    END AS price_band,
    COUNT(*)       AS product_count,
    ROUND(AVG(rating), 2)       AS avg_rating,
    ROUND(AVG(review_count), 0) AS avg_reviews
FROM bsr_top50
GROUP BY price_band
ORDER BY MIN(price);
```

如果发现 $250-400 区间商品少但需求大，这就是“价格真空带”——你的新品应该定在这里。

进阶玩法：用 UNION 对照「市场分布 vs 自家落点」

上面的 SQL 只能看到“市场榜单”一边的价格分布，光看分布学员还不知道“这跟我有什么关系”。把两张表合起来对照，张力立刻就出来了——左边是市场分布（找真空带），右边是自家 7 款落在哪些档（找差异化机会）：

```
-- 进阶版：两表对照（市场榜单分布 vs 自家 7 款落点）
SELECT '市场 TOP50' AS source,
       CASE WHEN price < 150 THEN '$0-150（低端）'
            WHEN price < 250 THEN '$150-250（中低端）'
            WHEN price < 400 THEN '$250-400（中端）'
            WHEN price < 600 THEN '$400-600（中高端）'
            ELSE '$600+（高端）' END AS price_band,
       COUNT(*) AS product_count
FROM bsr_top50
GROUP BY price_band
UNION ALL
SELECT '自家 7 款',
       CASE WHEN price < 150 THEN '$0-150（低端）'
            WHEN price < 250 THEN '$150-250（中低端）'
            WHEN price < 400 THEN '$250-400（中端）'
            WHEN price < 600 THEN '$400-600（中高端）'
            ELSE '$600+（高端）' END AS price_band,
       COUNT(*) AS product_count
```

```
FROM raw_products
GROUP BY price_band

ORDER BY price_band, source;
```

这样学员一眼就能看到：市场 TOP50 在 $250-400 档只有少数商品（真空带）→ 自家候选里恰好有 1 款落在这个档 → 这就是“差异化定价机会”。把“市场缺口”和“自家落点”放在同一张结果里，选品决策才能“一眼看明白”。

✅ 本次实验交付物

- 竞品差评痛点对比表
- 价格带分析结果
- 竞品分析报告（飞书文档）

实验八：N8N 工作流扩展——接续 Day01 串联全链路

- **时长：**45 分钟
- **目标：**把实验五、六、七手动做的分析接入 Day01 的 N8N 工作流，实现两天全链路一键打通
- **前置条件：**Day01 实验四的 N8N 工作流已搭建完成

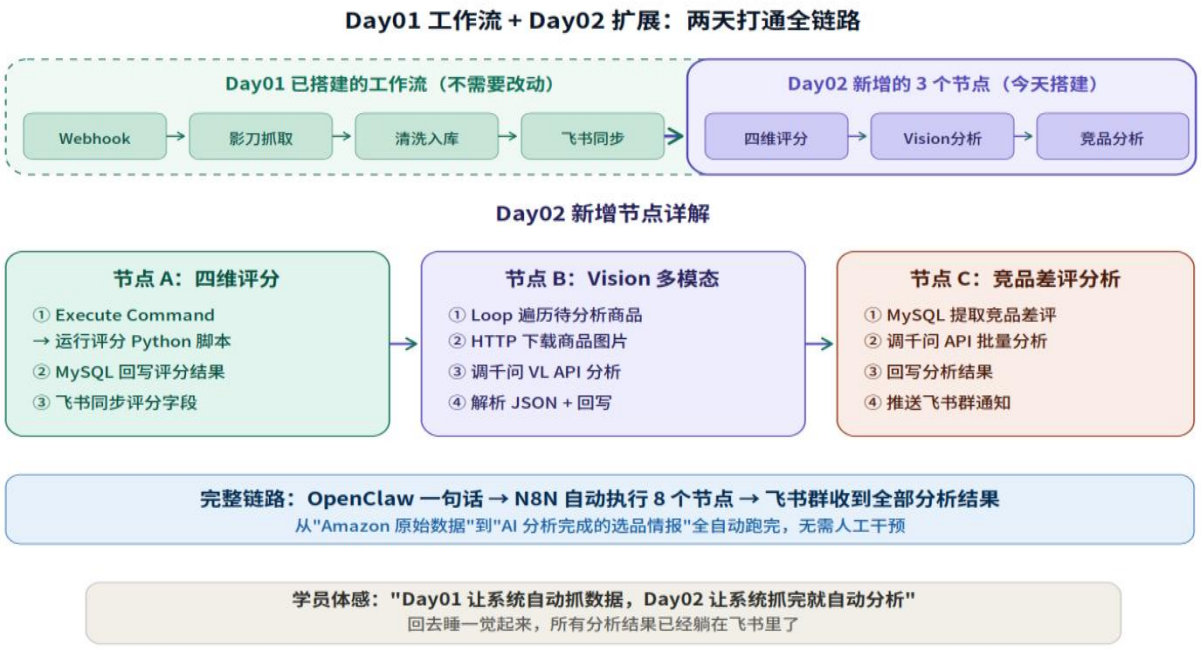
8.0 准备飞书多维表格？

接下来，需要将竞品有关的分析结果对外公开访问（给领导、同事、或业务口去看）。建议可以考虑利用飞书多维表格。访问并登录飞书（www.feishu.cn），继续在 Day01 中创建飞书多维表格：跨境选品池中新建一个数据表，重命名为：amazon 竞品评价分析表。结构如下：

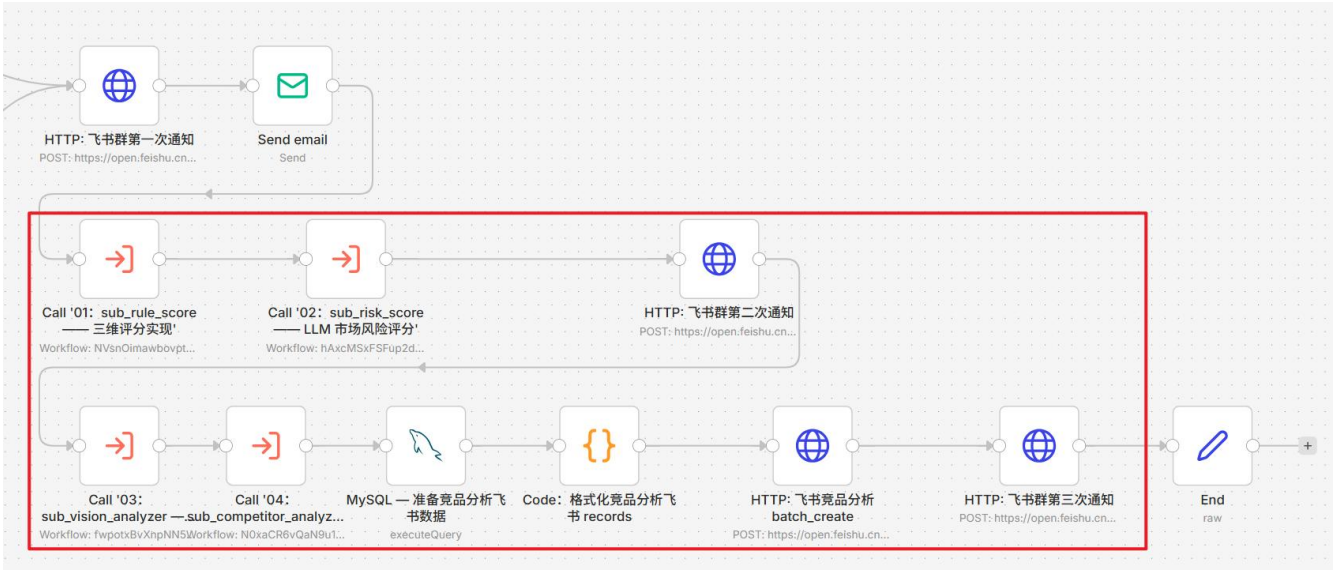


8.1 为什么要接续 Day01 的工作流？

前三个实验你手动做了三件事：手动跑评分脚本、手动触发 Vision 分析、手动跑竞品分析。如果每天都要这样操作一遍，还叫什么自动化？



整体新增工作流：



回忆 Day01 实验四：我们已经用 N8N 搭了一条工作流：Webhook 触发 → 影刀抓取 → 清洗入库 → 飞书同步。现在我们在这条工作流的末尾继续“接”三个新节点。

8.2 追加流程 A：四维评分自动化

在 Day01 工作流的“飞书同步”节点之后，添加一个 Execute Command 节点：

```
节点类型：Execute Command
节点名称：四维评分计算

命令：
    python3 /opt/scripts/scoring_engine.py
```

```
说明：
    这个脚本就是实验五中写的评分代码，
    打包成独立 .py 文件放到服务器上。
    脚本执行流程：
    ① 从 MySQL 读取新入库的商品数据
    ② 计算趋势热度分（SQL）
    ③ 计算竞争度分（SQL）
    ④ 计算利润空间分（Python）
    ⑤ 调用通义千问 API 计算市场风险分
    ⑥ 四维加权合成，写入 product_scores 表
    ⑦ 同步评分结果到飞书多维表格
```

无代码方案：直接复用前面章节的 N8N 工作流

> 📌 **重要说明**
> 本章节（8.2/8.3/8.4）用 Execute Command 节点调用 Python 脚本的原因是「图省事」——把前面写好的脚本直接调用一遍。但这并非唯一方案。如果你前面用的是无代码方案，可以把那些工作流「子工作流化」后接入 Day01 主流程，完全不需要 Python。

无代码替代方案：用 Execute Workflow 节点串联

N8N 提供了 Execute Workflow 节点，可以从一个工作流中调用另一个工作流。改造步骤：

步骤 1：把 5.2 节的「三维评分 N8N 工作流」保存为独立工作流，命名为 sub_rule_score。

步骤 2：把 5.3 节的「LLM 风险评分 N8N 工作流」保存为独立工作流，命名为 sub_risk_score。

步骤 3：把 6.4 节的「Vision 多模态 N8N 工作流」保存为 sub_vision_analyzer。

步骤 4：把 7.3 节的「差评分析 N8N 工作流」保存为 sub_competitor_analyzer。

步骤 5：在 Day01 主工作流的「飞书同步」节点后，依次添加 4 个 Execute Workflow 节点，分别引用上述 4 个子工作流。

注意：

1、使用前将这几个子工作流的第一个节点换成：**When Executed by Another Workflow 节点**

2、**将这个节点的 Input data mode 设置为：**Accept all data

3、必须将这个子工作流保存并**发布**，父工作流中才能正常调用

追加节点 1：Call '01：sub_rule_score —— 三维评分实现'

节点类型：Execute Sub-Workflow

Source：Database

Workflow：从列表中选择 [01：sub_rule_score —— 三维评分实现]

Mode：Run once with all items

Options：Wait For Sub-Workflow：开启（等待子工作流完成再继续）

追加节点 2：Call '02：sub_risk_score —— LLM 市场风险评分'

节点类型：Execute Sub-Workflow

Source：Database

Workflow：从列表中选择 [02：sub_risk_score —— LLM 市场风险评分]

Mode：Run once with all items

Options：Wait For Sub-Workflow：开启（等待子工作流完成再继续）

> **关键洞察**
> 本章 8.2/8.3/8.4 三个节点的核心是「让前面的分析能在 Day01 数据更新后自动跑」。无论用 Python 脚本还是 N8N 子工作流，达成的效果一样。但 Execute Workflow 方式让整个项目「保持纯 N8N 体系」，更符合「无代码 AI 应用开发」的课程定位。

追加节点 3：飞书群通知节点（HTTP：飞书群第二次通知）

```
节点类型：HTTP Request
节点名称：推送分析完成通知

方法：POST
URL：https://open.feishu.cn/open-apis/bot/v2/hook/{key}

请求体：
{
    "msg_type": "text",
    "content": {
        "text": "🤖 【AI 分析完成】\n"
              + "📋 已完成四维评分且已更新数据入库：\n"
              + "✅ 趋势热度评分完成\n"
              + "✅ 竞争度评分完成\n"
              + "✅ 利润空间评分完成\n"
              + "✅ 市场风险评分完成\n\n"
              + "接下来，将会进行:\n✔️ 1. Vision 多模态商品图片分析\n✔️ 2. 竞品深度解剖——差评挖掘与定价策略\n\n 请稍候.."
    }
}
```

真实配置：

URL：https://open.feishu.cn/open-apis/bot/v2/hook/9503e81e-5e38-4911-a80e-681d7746df35
JSON Body：

```
请求体：
{
    "msg_type": "text",
    "content": {
        "text": "🤖 【AI 分析完成】\n📋 已完成四维评分且已更新数据入库：\n✅ 趋势热度评分完成\n✅ 竞争度评分完成\n✅ 利润空间评分完成\n✅ 市场风险评分完成\n\n 接下来，将会进行:\n✅ 1. Vision 多模态商品图片分析\n✅ 2. 竞品深度解剖—差评挖掘与定价策略\n\n 请稍候...."
    }
```

```
}
```

8.3 追加流程 B：Vision 多模态与差评分析自动化

追加节点 4：Call '03： sub_vision_analyzer —— Vision 多模态 N8N 工作流'

节点类型：Execute Sub-Workflow

Source：Database

Workflow：从列表中选择 [03： sub_vision_analyzer —— Vision 多模态 N8N 工作流]

Mode：Run once with all items

Options：Wait For Sub-Workflow：开启（等待子工作流完成再继续）

追加节点 5：Call '04：sub_competitor_analyzer —— 差评批量分析'

节点类型：Execute Sub-Workflow

Source：Database

Workflow：从列表中选择 [04：sub_competitor_analyzer —— 差评批量分析]

Mode：Run once with all items

Options：Wait For Sub-Workflow：开启（等待子工作流完成再继续）

追加节点 6：MySQL — 准备竞品分析飞书数据节点

```
SELECT
    a.asin,
    a.brand AS 品牌,
    b.product_name,
    a.price AS 价格 USD,
    b.price_band AS 所处价位,
    b.trend_score AS 趋势热度分,
    b.competition_score AS 竞争度分,
    b.profit_score AS 利润空间分,
    b.risk_score AS 市场风险分,
    b.total_score AS 四维评分,
    a.vision_score AS 视觉评分,
    a.vision_audience AS 目标人群,
    a.vision_selling_pt AS 核心卖点,
    a.pain_points AS 商品痛点,
    a.pain_summary AS 痛点一句话总结
FROM raw_products a
INNER JOIN product_scores b
    ON a.asin = b.asin
WHERE a.crawl_date = CURDATE();    -- ← 加这一行，防止行膨胀
```

追加节点 7：Code：格式化竞品分析飞书 records 节点

Mode：Run Once for All Items

Language：JavaScript

JavaScript：

```
return [
  {
    json: {
      records: $input.all().map(item => ({
        fields: {
          "asin": item.json.asin,
          "品牌": item.json.品牌,
          "product_name": item.json.product_name,
          "价格 USD": Number(item.json.价格 USD),
          "所处价位": item.json.所处价位,
          "趋势热度分": Number(item.json.趋势热度分),
          "竞争度分": Number(item.json.竞争度分),
          "利润空间分": Number(item.json.利润空间分),
          "市场风险分": Number(item.json.市场风险分),
          "四维评分": Number(item.json.四维评分),
          "视觉评分": Number(item.json.视觉评分),
          "目标人群": item.json.目标人群,
          "核心卖点": item.json.核心卖点,
          "商品痛点": item.json.商品痛点,
          "痛点一句话总结": item.json.痛点一句话总结,
          "创建时间":  Date.now()
        }
      }))
    }
  }
];
```

追加节点 8：HTTP：飞书竞品分析 batch_create

```
节点类型：HTTP Request
节点名称：HTTP：飞书竞品分析 batch_create
方法：POST
URL：https://open.feishu.cn/open-
apis/bitable/v1/apps/{app_token}/tables/{table_id}/records/batch_create
Header Parameters:
Name：Authorization
Value：Bearer {{ $('HTTP：获取飞书 Token').first().json.tenant_access_token }}

Name：Content-Type
Value：application/json

Send Body：启动
Body Content Type：JSON
Specify Body：Using JSON
JSON:{{ $json }}
```

追加节点 9：飞书群通知节点（HTTP：飞书群第三次通知）

```
节点类型：HTTP Request
节点名称：HTTP：飞书群第三次通知

方法：POST
URL：https://open.feishu.cn/open-apis/bot/v2/hook/9503e81e-5e38-4911-a80e-
```

```
681d7746df35

请求体：
{
  "msg_type": "text",
  "content": {
    "text": "📊 今日竞品数据分析完成！\n✅ 四维评分 \n✅ Vision 多模态商品图分析\n✅ 竞品的差评挖掘\n✅ 定价策略\n\n 以上均已分析完毕并写入 MySQL 和飞书多维表格。\n\n📋 查看竞品分析评价：
\nhttps://my.feishu.cn/base/PQ99bmqzpap86hs9S3KcLMf9nEf?from=from_copylink\n\n 即将生成可视化竞品分析大屏看板与 amazon 选品数据分析报告，请稍候......"
  }
}
```

以上【从零搭建跨境电商情报 N8N 采集系统】保存并发布！、

8.4 OpenClaw 全链路测试

现在回到 OpenClaw，输入“**启动今日数据采集**”，观察整个链路自动执行：

```
执行顺序（全部自动，无需人工干预）：

Day01 部分（约 5-10 分钟）：
    ① Webhook 接收指令
    ② 影刀 RPA 抓取 Amazon 数据 → 保存 Excel
    ③ Python 清洗 → 写入 MySQL
    ④ 飞书多维表格同步

Day02 部分（约 10-15 分钟）：
    ⑤ 四维评分引擎计算 → 结果回写
    ⑥ Vision 多模态分析商品图片 → 结果回写
    ⑦ 竞品差评批量分析 → 生成报告
    ⑧ 飞书群推送"分析完成"通知

总耗时约 15-25 分钟，完全无人值守
```

教学高潮

这是 Day02 的“大结局”——Day01 让系统自动抓数据，Day02 让系统抓完就自动分析。学员回去睡一觉起来，所有分析结果已经躺在飞书里了，飞书群也收到了通知。

✅ 本实验交付物

- Day01 N8N 工作流成功扩展 3 个新节点
- 全链路 8 个节点一键执行测试通过
- 飞书群收到“AI 分析完成”通知
- Day01 + Day02 两天全链路打通

实验九：飞书 AI 快捷字段配置

时长：30 分钟　**目标：**用飞书原生 AI 能力实现轻量级智能标注

9.1 三种 AI 能力接入方式对比

今天我们已经用了 N8N + LLM API 做重度分析。现在用飞书 AI 快捷字段做轻量级标注，体会三种工具的差异：



9.2 配置三个 AI 字段

字段一：AI 翻译（英文标题 → 中文营销话术）

在飞书多维表格中新增字段：

```
字段名：商品标题
字段类型：自定义 AI 自动填充
输入引用：「product_name」字段
Prompt：
    将以下英文商品标题：product_name  翻译为中文，并改写为适合电商平台的营销话术，突出核心卖点，不超过 30 字。
```

字段二：AI 标签（自动打标爆款/潜力/高风险）

```
字段名：选品标签
字段类型：自定义 AI 自动填充
输入引用：「四维评分」「视觉评分」「所处价位」「目标人群」字段
Prompt：
        已知商品四维评分为：  ，图片视觉评分：  ，商品所处价位：  ，目标人群：  。
    请根据以上商品数据判断选品等级：
        - 「爆款」
        - 「潜力」
```

- 「高风险」
- 其他：标记为「观望」

仅返回标签文字。



字段三：营销宣传话术（一句话提炼卖点）

字段名：营销宣传话术
字段类型：自定义 AI 自动填充
输入引用：「product_name」「四维评分」「所处价位」「目标人群」「核心卖点」字段
Prompt：
请根据 「product_name」「四维评分」「所处价位」「目标人群」「核心卖点」 生成最关于本商品的对外宣传售卖话术，要提炼产品卖点，紧扣用户需求，话术要有吸引力和冲击力。

教学对比点
飞书 AI 字段 5 分钟就配好了，但它不能自定义模型、不能强制 JSON 输出、不能做复杂多步推理。所以轻量辅助用飞书，重度分析用 N8N + LLM API。

✅ 本实验交付物

- 飞书多维表格新增 3 个 AI 字段，自动填充完成
- 理解了三种 AI 接入方式的适用场景

Day02 总结

今天你完成了以下工作：

实验	完成了什么	产出物
实验五	四维评分引擎	全部商品有了综合得分和排名
实验六	Vision 多模态分析	7 款商品视觉数据 + 竞争力矩阵
实验七	竞品差评挖掘	痛点对比表 + 价格带分析 + 策略报告
实验八	N8N 全链路串联	Day01+Day02 两天 8 个节点一键打通
实验九	飞书 AI 字段	3 个 AI 字段自动填充完成

Day03 预告

Day03 我们将把所有分析结果变成可视化看板，用 OpenClaw 自动生成每日选品日报推送飞书群，并整理项目复盘文档和面试话术。数据将从“看得懂”变成“自动跑”。